

SCSSE01I

Object Oriented Design



Module Leader Prof. Abeer Hamdy

Supply Chain Tracking System

G11



Phase 3

By:

Youssef AlaaElDin Mohamed	241784	A2
Mostafa Hossam	240503	A2
Zeyad Ashraf	245975	A3
Ziad Sameh	233802	A2
Ziyad Mohamed	243886	A2

Contents

1. System Description (Group Work)	3
1.1 Introduction	3
1.2 System Objectives	3
1.3 Stakeholders	3
Admin.....	4
Manufacturer	4
Shipper.....	4
Warehouse Manager	4
Retailer.....	4
1.4 Functional Requirements	4
1.5 Non-Functional Requirements	5
1.6 Assumptions and Constraints	5
2. Use case diagram (Group Work).....	5
3. Use case scenarios in detailed format (Individual).....	6
4. System sequence diagrams (Individual Work)	16
5. Full Class Diagram	27
6. Design Patterns	27
Abstraction-Occurrence Design Pattern	28
Strategy Design Pattern	28
Immutable Design Pattern	29
Observer design pattern:	30
Read only design pattern:	30
7. Sequence Diagrams	31
8. CRC Cards	39
9. Testing Reports	40
Report 1	40
Report 2	40
Report 3	40

Name	Contribution
Youssef AlaaElDin Mohamed	UC1, UC2 Scenario and SSD & SD, Class Diagram, Use Case Diagram, Abstraction-Occurrence Design Pattern • Cycle 1: Good.java -> GoodControl.java -> ManufacturerGoodsManagementUI.Java • Cycle 2: Item.java -> InventoryControl.java -> WarehouseInventoryManagementUI.java • Cycle 3: User.java -> AuthenticationControl.java -> LoginUI.java ○ Others: Manufacturer.java, Warehouse.java, WarehouseManager.java, Retailer.java, Role.java, Navigation.java, UiKit.java, DashboardFrame.java, ... Exe file
Mostafa Hossam	UC3, UC4 Scenario and SSD & SD, Use Case Diagram, Read-Only Pattern • Cycle: Shipper.java -> VehicleControl.java -> VehicleManagementUI.java ○ Others: ReadOnlyShipment.java, DelayEvent.java, FailedDeliveryEvent.java
Zeyad Ashraf	UC5, UC6 Scenario and SSD & SD, Use Case Diagram, Strategy Pattern • Cycle: Event.java -> EventControl.java -> EventRecordingFrame.java ○ Others: InspectionStrategy.java, StandardInspectionStrategy.java, FragileGoodsInspectionStrategy.java, HazardousInspectionStrategy.java, InspectionEvent.java
Ziad Sameh	UC7, UC8 Scenario and SSD & SD, Use Case Diagram, Immutable pattern • Cycle: Admin.java -> AdminControl.java -> AdminUI.java ○ Others: Event.java, DispatchEvent.java, DeliveryEvent.java, DeliveryConfirmationEvent.java, ReturnRequestedEvent.java, ReturnDispatchedEvent.java, ReturnedEvent.java, DeliveryEvidence.java, Report.java
Ziyad Mohamed	UC9, UC10 Scenario and SSD & SD, Use Case Diagram, Observer Pattern • Cycle: Shipment.java -> ShipmentControl.java -> ShipmentManagementUI.java

	○ Others: ShipmentObserver.java, AdminObserver.java, RetailerObserver.java, WarehouseObserver.java, ShipmentCreatedEvent.java, ShipmentAcceptanceEvent.java
--	---

1. System Description (Group Work)

1.1 Introduction

The Supply Chain Tracking System (SCTS) is a desktop software application designed to model and track the movement of goods across a supply chain involving four types of independent organizations: Manufacturers, Shippers, Warehouses, and Retailers.

Modern supply chain systems have multiple stakeholders and hard logistics. Companies can face issues such as delayed shipments, inaccurate inventory records, and poor communication between parties. All that is because of poor non-assigned tracking systems. The proposed system provides a centralized digital platform to address these challenges.

An important design principle for the system is an event driven approach: every action performed on a good or shipment such as production, dispatch, receipt, inspection, storage, or delivery is recorded as a fixed event object. The system identifies the status of any good or shipment from its complete chronological event history, instead of storing status as a direct attribute. This ensures full tracking and verification across the supply chain and ensures that they are not altered.

1.2 System Objectives

- Track the movement of goods across the supply chain through recorded, immutable events.
- Maintain a complete and auditable event history for every good and shipment.
- Derive shipment status dynamically from event history rather than direct updates.
- Enforce role-based access so each stakeholder only views and acts on relevant data.
- Generate operational reports and event timeline views for authorized stakeholders.

1.3 Stakeholders

Admin

- Manages system users
- Generates reports

Manufacturer

- Registers newly produced goods into the system.
- Creates shipments for the produced goods, and assigns shippers to these shipments.
- Records production events.

Shipper

- Records dispatch events for the shipments assigned to them.
- Records delivery events for the shipments assigned to them.
- Records delay events for the shipments assigned to them.
- Responsible for recording events throughout the shipping process.

Warehouse Manager

- Confirms receipt of shipments at the warehouse.
- Records inspection and storage events.
- Updates warehouse inventory.

Retailer

- Receive goods from shippers or warehouses.
- Confirms and records delivery of goods.
- Views delivery timelines and event histories.

1.4 Functional Requirements

1. Allow the administrator to create, update, and deactivate user accounts.
2. Allow manufacturers to register new goods with unique identifiers and attributes.
3. Allow manufacturers to create shipments and assign them to a shipper.

4. Allow shippers to record dispatch and in-transit events for their assigned shipments.
5. Allow warehouse managers to confirm receipt of shipments and record inspection and storage events.
6. Allow retailers to confirm and record delivery of goods.
7. Derive and display the current status of any good or shipment from its event history.
8. Display a full event timeline for any good or shipment to authorized stakeholders such as admin.
9. Prevent modification or deletion of any recorded event (immutability).
10. Generate inventory and shipment reports for authorized stakeholders.

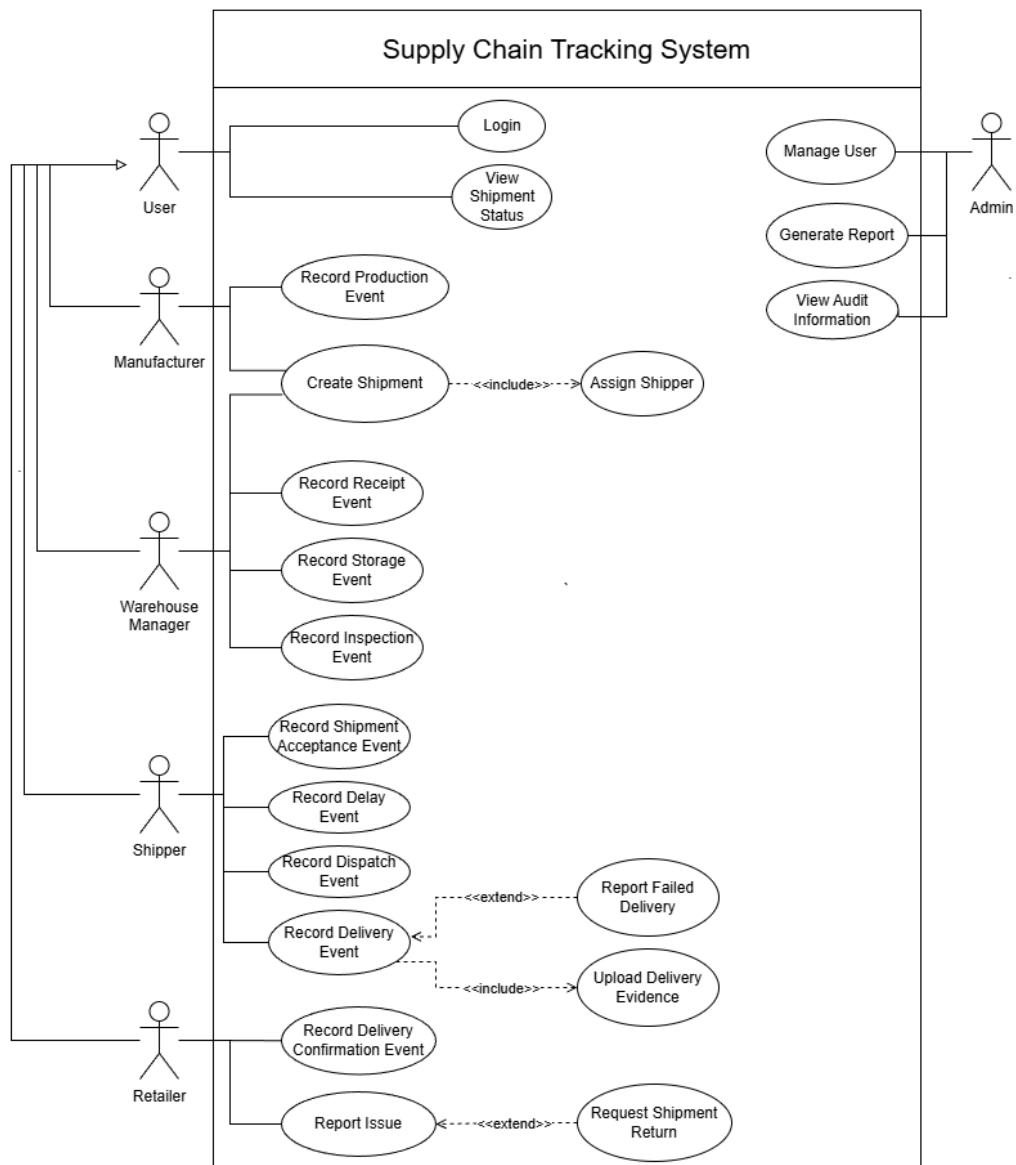
1.5 Non-Functional Requirements

1. The system must authenticate all users before granting access and enforce role-based access control at every operation.
2. The system must ensure no event data is lost; all recorded events are backed up immediately upon creation.
3. Timeline queries for any good or shipment must return results within 2 seconds.
4. The interface must be intuitive and require minimal training for stakeholders to perform core tasks.
5. The system must always ensure data consistency and reliability.

1.6 Assumptions and Constraints

1. Each good has a unique product ID and each shipment has a unique tracking number.
2. System is considered as a desktop application; furthermore, web and mobile interfaces are out of scope.
3. Stakeholders or users are pre-registered by the admin before they can log in.
4. Events cannot be modified or deleted after recorded to ensure no altering is done.
5. There is no payment processing included in the scope of the system.

2. Use case diagram (Group Work)



3. Use case scenarios in detailed format (Individual)

1. Use Case 1:

ID:	UC_01
Title:	Create Shipment
Description:	The Manufacturer has produced goods, and wants to initiate the shipment process; furthermore, the manufacturer decides to create a new shipment record in the Supply Chain Tracking System.
Primary Actor:	Manufacturer / Warehouse Manager
Secondary Actor:	N/A
Preconditions:	• Manufacturer / Warehouse Manager is logged in successfully (authenticated) • Manufacturer / Warehouse Manager is authorized to create shipments • The goods that are required to be shipped actually exist in the system and available (produced and not in another active shipment). • The system has at least one shipper registered.
Postconditions:	• A new Shipment record is created that has a unique Shipment ID • An immutable event “Shipment Created” is recorded and linked to the shipment. • A shipper is assigned to the shipment. • Shipment status becomes available to view in “View Shipment Status”
Main Success Scenario:	1. Manufacturer / Warehouse Manager wants to create a shipment, so they press on Create Shipment. 2. Manufacturer / Warehouse Manager fills a system prompt for shipment details that includes some information such as destination, pickup location, cargo). 3. Manufacturer / Warehouse Manager selects the goods that they want to ship. 4. Manufacturer / Warehouse Manager is then requested to choose a shipper (include: Assign Shipper) 5. Manufacturer / Warehouse Manager presses on Confirm Shipment. 6. Manufacturer / Warehouse Manager is displayed a confirmation message that includes the Shipment ID and the event “Shipment Created”.
Extensions:	2a. Missing or invalid fields are highlighted in the prompt, and the manufacturer / warehouse manager fills them in again.

	3a. The goods selected are unavailable (already assigned to another shipment); therefore, the manufacturer / warehouse manager is informed and then asked to select different goods. 4a. There are no available shippers, having the manufacturer / warehouse manager be informed that the shipment cannot be created unless a shipper is available.
Priority:	High

2. Use Case 2:

ID:	UC_02
Title:	Record Delivery Event
Description:	The shipper transports the shipment to the required destination, and is then required to confirm the completion of the delivery, record the event of delivery, and provide delivery details such as the time of delivery. Furthermore, this is part of maintaining the shipment's immutable event history.
Primary Actor:	Shipper
Secondary Actor:	N/A
Preconditions:	• The shipper is successfully logged in the system (authenticated) • The shipment record exists inside the system. • The shipment has already been dispatched. • The shipper is authorized and is the assigned shipper to that chosen shipment.
Postconditions:	• A delivered event that is linked to the shipment is recorded inside the system. • Shipment status is updated to "Delivered Successfully". • Delivery timestamp is recorded and stored inside the system. • The delivery can be failed or successful; furthermore, in both cases the shipper is required to upload delivery evidence and it is kept for following up with any issues. (Include: Upload Delivery Evidence).
Main Success Scenario:	1. The shipper drops off the shipment and navigates to the assigned shipment details page. 2. The shipper then presses on Record Delivery Event.

	3. The shipper fills a prompt to enter delivery details such as receiver name. 4. The shipper is then requested to upload delivery evidence and uploads a valid photo/video evidence (include: Upload Delivery Evidence). 5. The shipper clicks on Confirm Delivery. 6. The shipper is then displayed a confirmation message and a “Delivered” event is now visible & attached to the shipment.
Extensions:	3a. Missing or invalid fields are highlighted in the prompt, and the manufacturer / warehouse manager fills them in again. 4a. Delivery evidence is invalid therefore the manufacturer / warehouse manager has to upload another photo/video. 5a. Shipment is already marked as delivered; therefore, duplicate recording is prevented and shipper is informed and redirected back. 6a. The delivery fails by shipper’s decision or the customer’s refusal; therefore, the shipper selects “Report Failed Delivery” (extends: Report Failed Delivery) and is marked for further review.
Priority:	High

3. Use Case 3:

ID:	UC03
Title:	Record delivery confirmation event
Description:	Allows the retailer to confirm that a shipment has successfully been delivered by recording a delivery confirmation event in the system
Primary Actor:	Retailer
Secondary Actor:	None
Preconditions:	-Retailer is logged in -Shipment is identified -Shipment status delivered or waiting confirmation
Postconditions:	-Delivery confirmation event is recorded -Shipment status is updated
Main Success Scenario:	1- Retailer selects the shipment 2- System displays shipment details 3- Retailer reviews the shipment 4- System displays delivery confirmation form 5- Retailer enters delivery confirmation details 6- System validates the details

	7- Retailer confirms the delivery 8- System records the delivery confirmation event 9- System updates the shipment status
Extensions:	4a) Missing delivery details - System shows error - Retailer re enters details
Priority:	High

4. Use Case 4:

ID:	UC04
Title:	Report issue
Description:	Allows the retailer to report a problem related to a shipment and request a shipment return
Primary Actor:	Retailer
Secondary Actor:	None
Preconditions:	-Retailer is verified -shipment exists in the system -Shipment status is delivered
Postconditions:	-An issue event is recorded -Shipment status is updated -If selected, return request event is recorded
Main Success Scenario:	1- Retailer selects the shipment 2- System displays shipment details 3- Retailer reviews the shipment 4- Retailer enters issue details 5- System validates the details 6- Retailer submits the issue report 7- System records the issue event 8- System updates the shipment status
Extensions:	4a) Invalid issue message -Systems displays error -Retailer enters correct details
Priority:	High

5. Use Case 5:

ID:	UC05
------------	-------------

Title:	Record Inspection Event
Description:	The Warehouse Manager inspects incoming shipments to check their status before the goods proceed through supply chain operations. The inspection outcomes are recorded as immutable events.
Primary Actor:	Warehouse Manager
Secondary Actor:	N/A
Preconditions:	• Warehouse Manager is logged into the system. • The shipment exists in the system with a shipment ID. • A Record Receipt Event has already been recorded for the shipment, confirming it has arrived at the warehouse.
Postconditions:	• An immutable inspection event is recorded and linked to the shipment. • Shipment status is updated to either Inspected Approved or Inspected Rejected. • The inspection event is archived in the audit trail.
Main Success Scenario:	1. The Warehouse Manager selects Record Inspection Event from the menu. 2. The Warehouse Manager searches through the list of shipments received and selects the wanted shipment. 3. The Warehouse Manager enters the inspection details, including the inspection date, inspection status (Approved / Damaged / Rejected), and inspection notes. 4. The Warehouse Manager submits the inspection details. 5. The Warehouse Manager receives confirmation that the inspection event is successfully recorded.
Extensions:	2a. The Warehouse Manager is searching for a shipment that doesn't exist. a. The Warehouse Manager is shown an error message indicating the shipment was not found.

	3a. The Warehouse Manager submits inspection details with missing or invalid fields. a. The Warehouse Manager is alerted to correct the invalid or missing information. 3b. The Warehouse Manager sets the inspection status to Damaged or Rejected. a. The Warehouse Manager is shown a warning that the shipment has been flagged and will be blocked from proceeding to dispatch.
Priority:	High

6. Use Case 6:

ID:	UC06
Title:	Record Dispatch Event
Description:	The Shipper records a dispatch event when an approved shipment departs from the warehouse to begin its delivery route. The event is stored as an immutable record, and the shipment status is updated to In Transit.
Primary Actor:	Shipper
Secondary Actor:	N/A
Preconditions:	• Shipper is logged into the system. • The shipment exists in the system and is assigned to this Shipper. • The shipment status is Inspected - Approved
Postconditions:	• An immutable dispatch event is recorded and linked to the shipment. • Shipment status is updated to In Transit. • Dispatch details are saved to audit history.

Main Success Scenario:	1. The Shipper selects Record Dispatch Event from the menu. 2. The Shipper views the list of shipments ready for dispatch and selects the relevant shipment. 3. The Shipper enters the dispatch details, including the dispatch date, dispatch time, and vehicle and transport information. 4. The Shipper ends the dispatch process by submitting the dispatch record. 5. The Shipper receives a confirmation message that the dispatch event was recorded, and the shipment status has changed to In Transit.
Extensions:	2a. The Shipper selects a shipment that has not been approved after inspection. a. The Shipper receives a warning that the shipment is not approved for dispatch and cannot proceed. 3a. The Shipper submits dispatch details with missing or invalid information. a. The Shipper is asked to correct the missing or invalid fields before submitting. 4a. A system error occurs during the recording of the dispatch event. a. The Shipper is notified that the dispatch event failed to be recorded and is asked to try again.
Priority:	High

7. Use Case 7: Record Production Event

ID:	UC07
Title:	Record Production Event

Description:	The Manufacturer records a production event for a newly produced good.
Primary Actor:	Manufacturer
Secondary Actor:	N/A
Preconditions:	○ Manufacturer is authenticated. ○ Manufacturer is authorized to record production events. ○ The system is operational.
Postconditions:	○ A Production Event is permanently stored. ○ The good is registered and linked to its event history. ○ The current status of the good can be derived as Produced. ○ The created event cannot be modified or deleted.
Main Success Scenario:	1- Manufacturer selects Record Production Event . 2- Manufacturer enters the production details. 3- Manufacturer submits the production event.
Extensions:	○ 2a. Manufacturer enters missing or invalid data. ○ 2a1. Manufacturer corrects the data and resubmits. ○ 3a. Manufacturer enters a duplicate Good ID. ○ 3a1. Manufacturer re-enters a valid Good ID and submits again. ○ 3b. Manufacturer cancels the process. ○ 3b1. Use case ends without recording the event.
Priority:	High

8. Use Case 8: Record Shipment Acceptance Event

ID:	UC08
Title:	Record Shipment Acceptance Event
Description:	The Shipper records an acceptance event confirming responsibility for transporting a shipment.
Primary Actor:	Shipper
Secondary Actor:	N/A
Preconditions:	○ Shipper is authenticated. ○ Shipment exists in the system. ○ Shipment is assigned to the Shipper.
Postconditions:	○ Shipment Acceptance Event is permanently stored. ○ Shipment status can be derived as Accepted by Shipper. ○ The shipment timeline is updated. ○ The created event cannot be modified or deleted.
Main Success Scenario:	1- Shipper views assigned shipments. 2- Shipper selects a shipment.

	3- Shipper chooses Accept Shipment . 4- Shipper confirms acceptance.
Extensions:	○ 1a. Shipper has no assigned shipments. ○ 1a1. Use case ends. ○ 2a. Shipper selects a shipment that is not assigned to them. ○ 2a1. Action is denied. ○ 3a. Shipper selects a shipment that has already been accepted. ○ 3a1. Acceptance is not completed. ○ 4a. Shipper cancels the acceptance process. ○ 4a1. Use case ends without recording the event.
Priority:	High

9. Use Case 9:

ID:	UC09
Title:	Record Receipt Event
Description:	This case allows the Warehouse to record the receipt of a shipment that record the arrival time and checks the status of the shipment.
Primary Actor:	Warehouse Manager
Secondary Actor:	none
Preconditions:	• Shipment exists in the system. • Warehouse Manager is logged into the system.
Postconditions:	• Receipt event is recorded as an immutable event. • Shipment timeline is updated. • Shipment status is delivered as (received at warehouse).
Main Success Scenario:	1. Warehouse select record receipt event. 2. Warehouse enters shipment ID. 3. Warehouse Manager verifies the shipment details 4. Warehouse confirms goods received (quantity, condition).
Extensions:	2a. Shipment not found System displays error message. (Warehouse re enters valid shipment ID). 4a. Quantity didn't match or damaged goods
Priority:	High

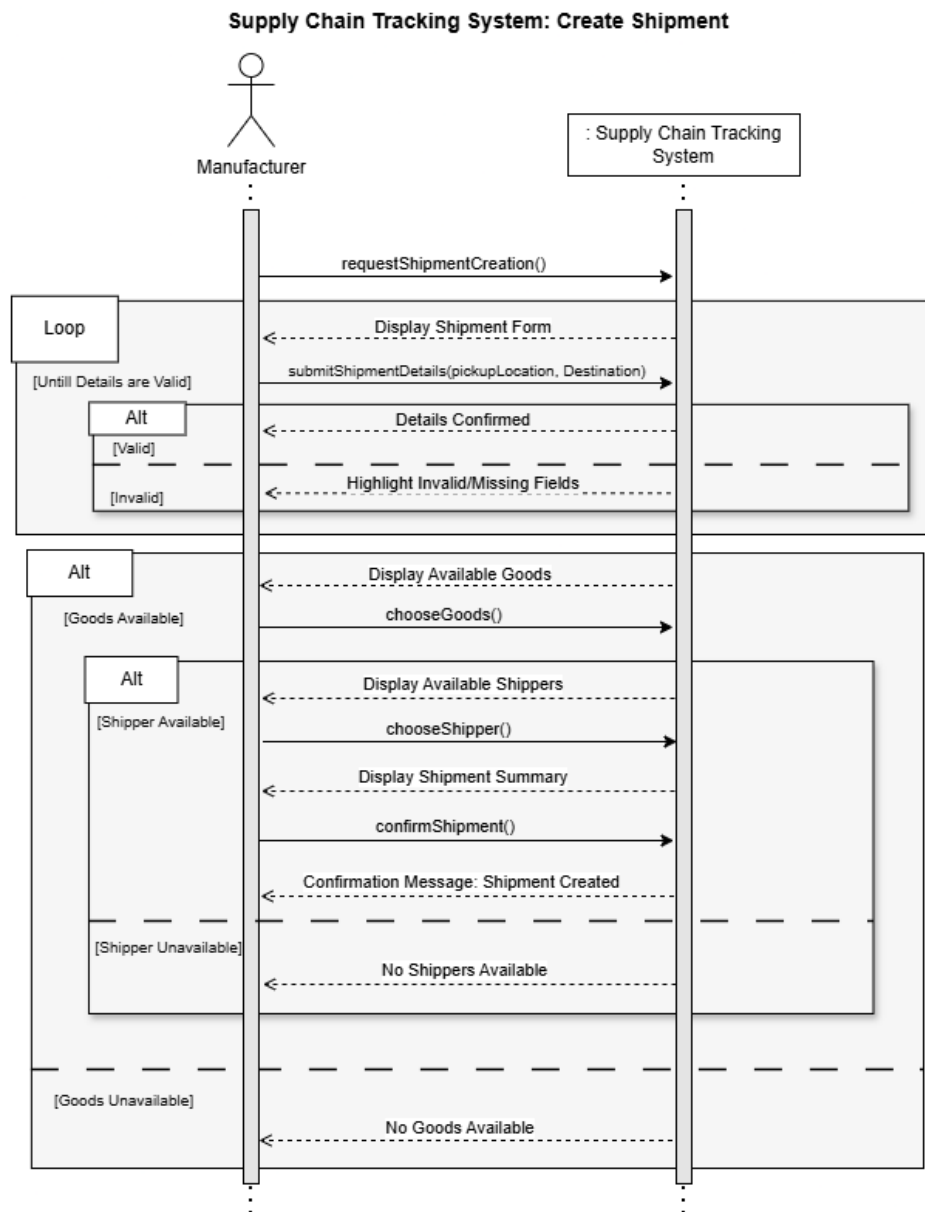
10. Use Case 10:

ID:	UC10
Title:	Record Storage Event
Description:	This use case allows the Warehouse to record that received goods have been stored in a specific storage location.
Primary Actor:	Warehouse Manager
Secondary Actor:	N/A
Preconditions:	• Shipment has been received. • Receipt Event already exists.
Postconditions:	• Storage Event is recorded. • Storage location is associated with goods. • Shipment timeline is updated.
Main Success Scenario:	1. Warehouse selects a received shipment. 2. Warehouse chooses Record Storage Event. 3. Warehouse prompts for storage details (location, quantity, temperature). 4. Warehouse enters storage information.
Extensions:	3a. Invalid storage location • System displays validation errors. • Warehouse corrects location details. 1a. Shipment not yet received • System prevents storage recording. • Displays message: "Receipt Event required before Storage."
Priority:	Medium

4. System sequence diagrams (Individual Work)

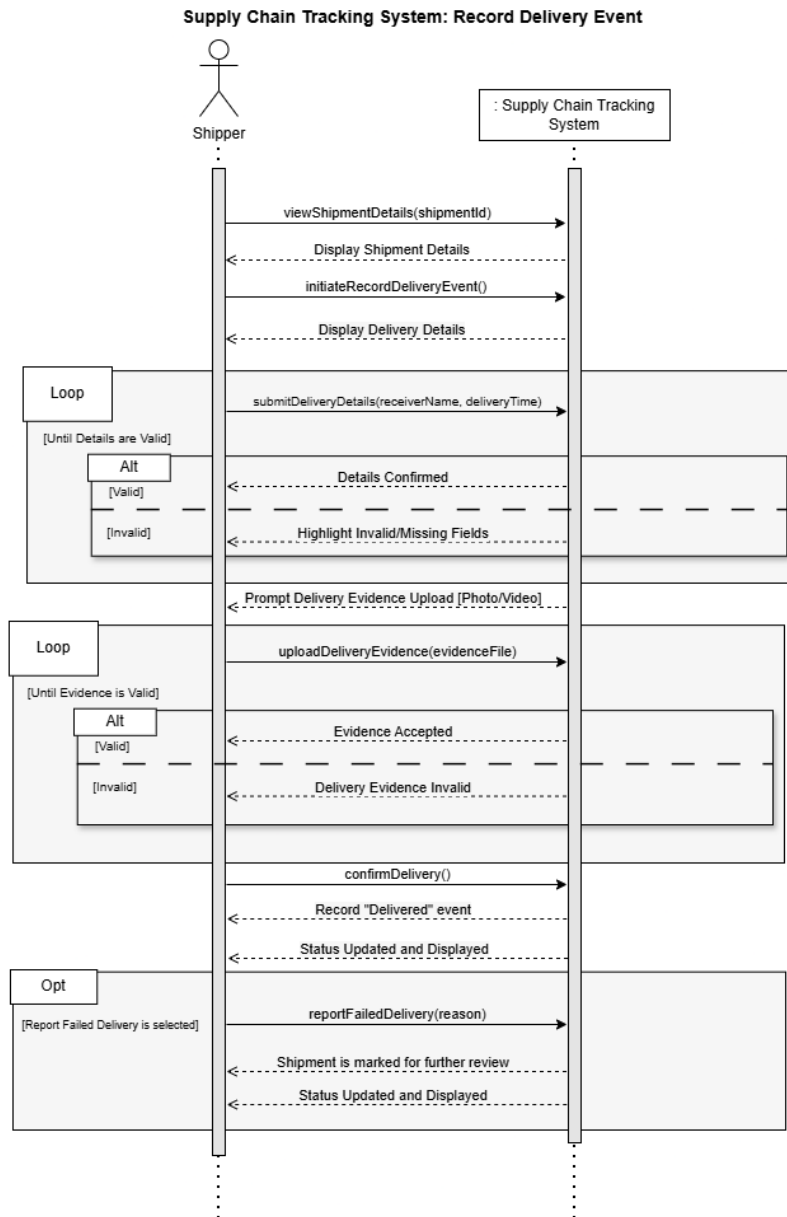
1. System Sequence diagram 1:

Create Shipment (UC01)

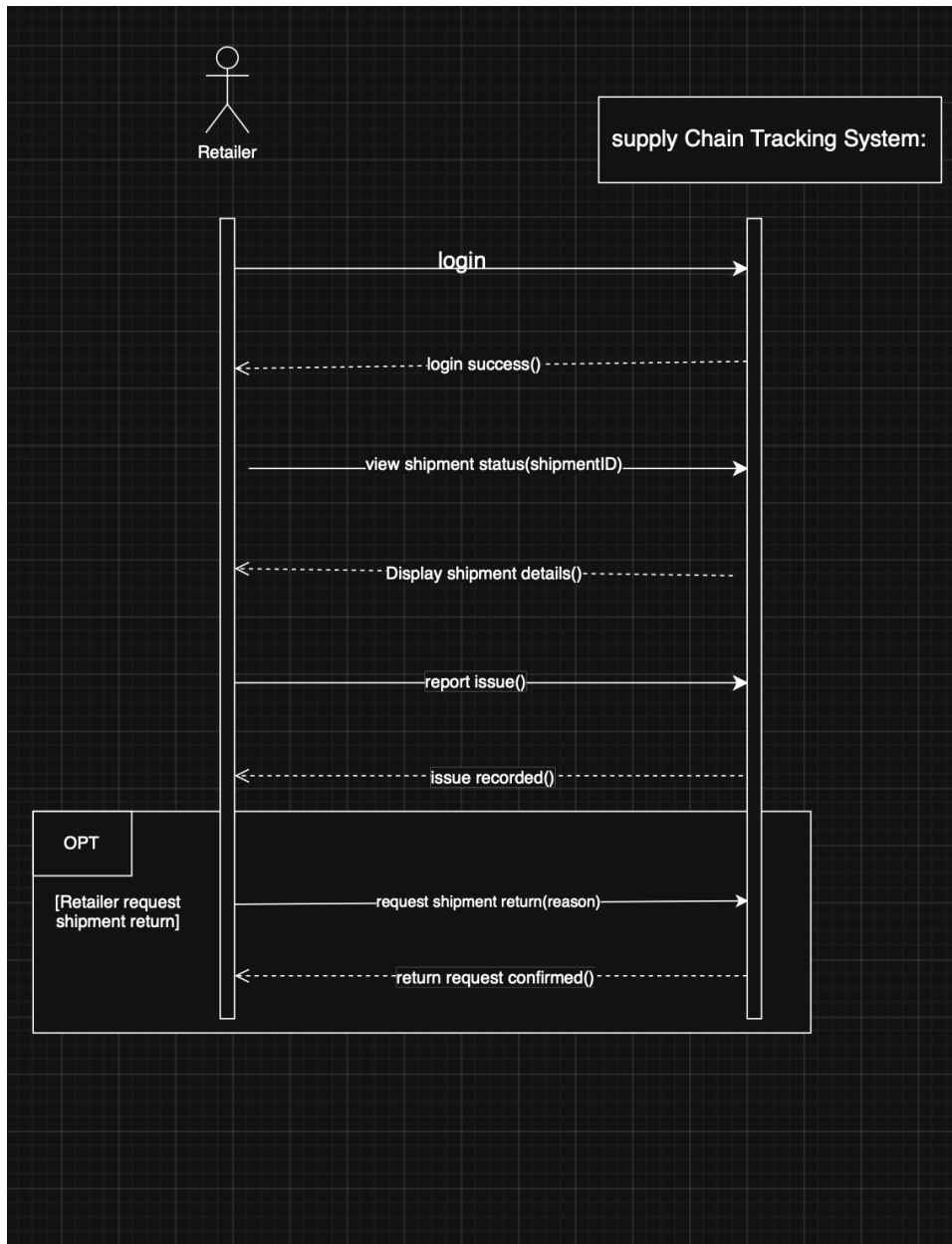


2. System Sequence diagram 2:

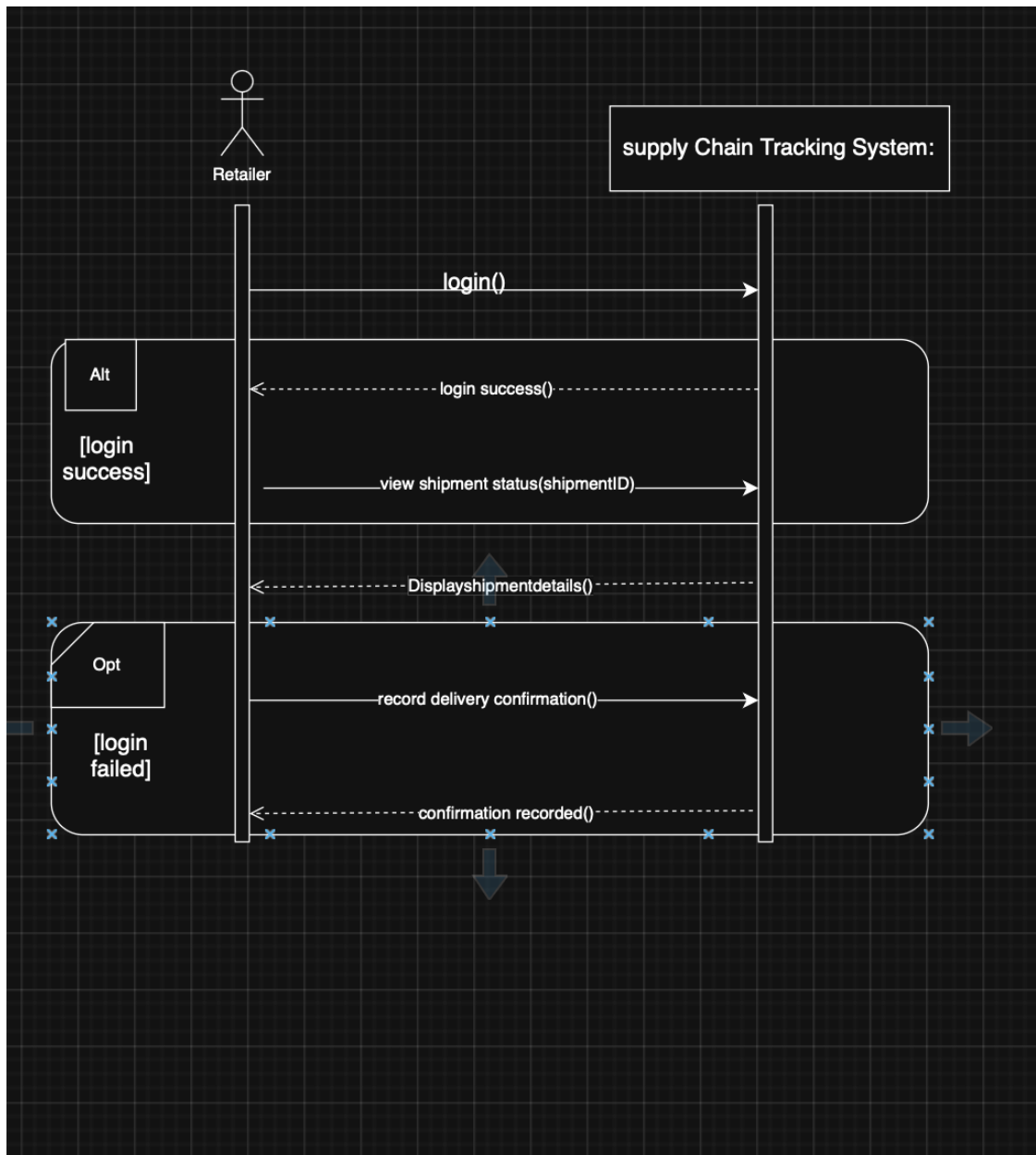
Record Delivery Event (UC02)



3. System Sequence diagram 3:

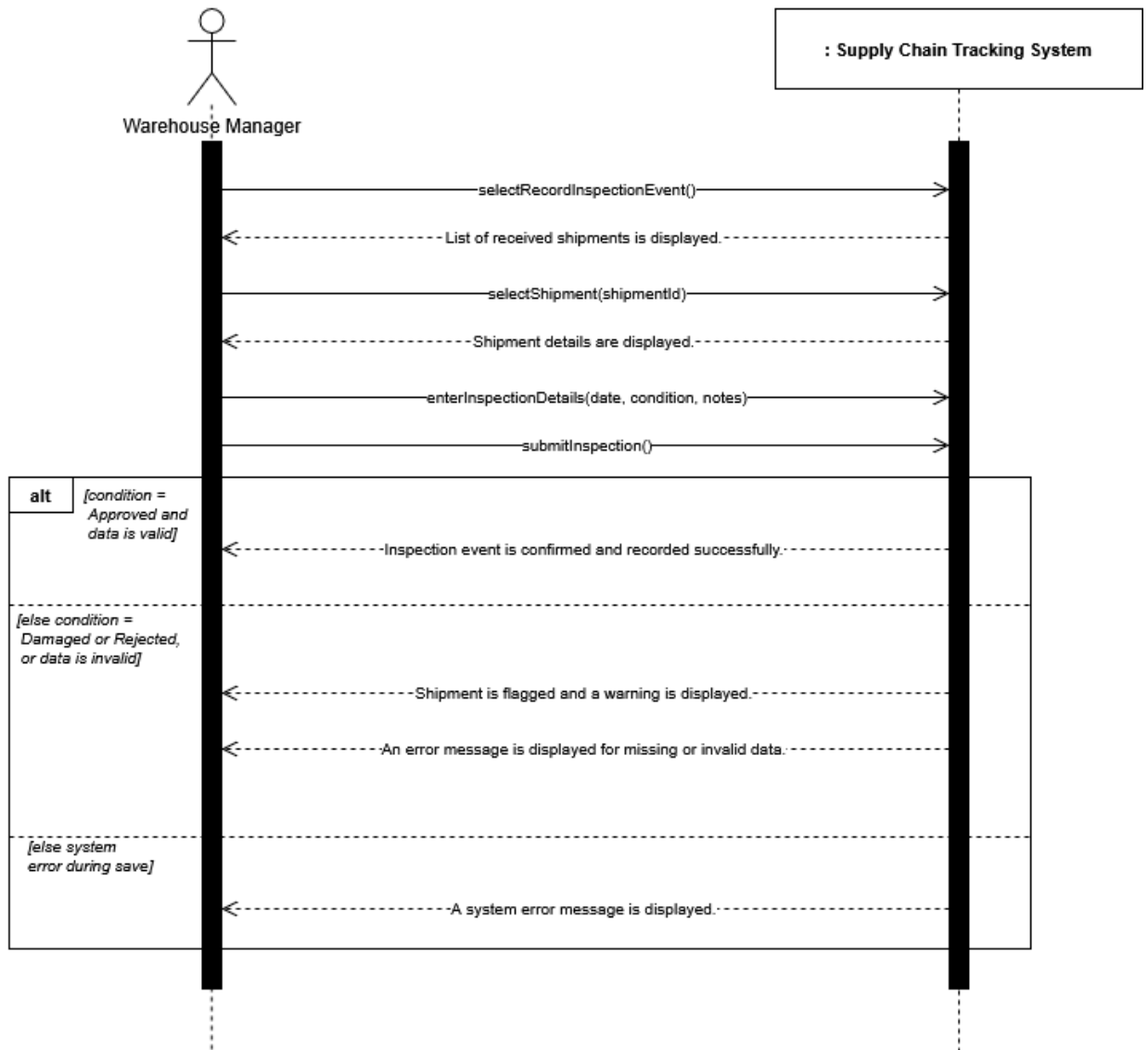


4. System Sequence diagram 4:



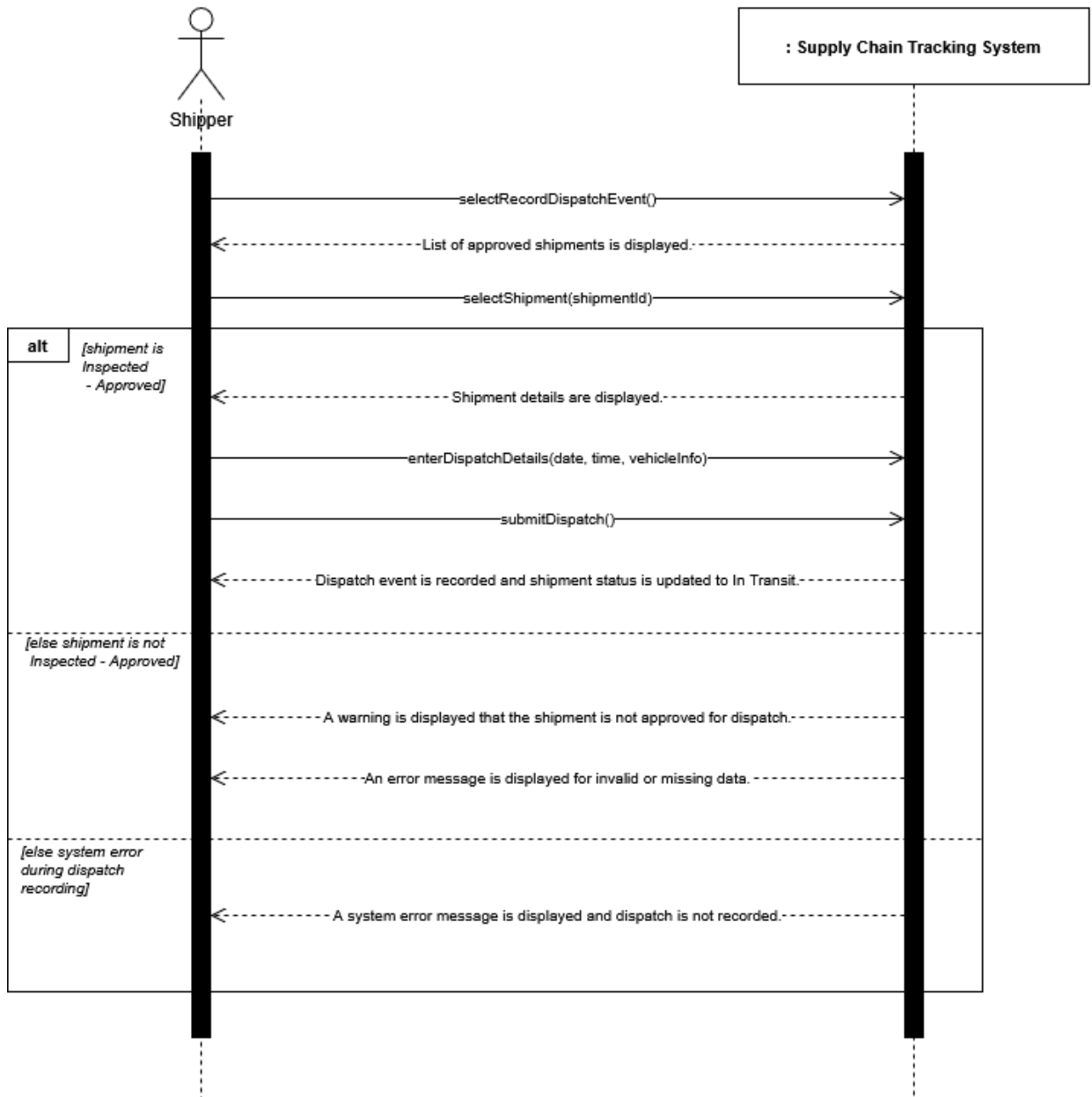
5. System Sequence diagram 5:

Supply Chain Tracking System: Record Inspection Event

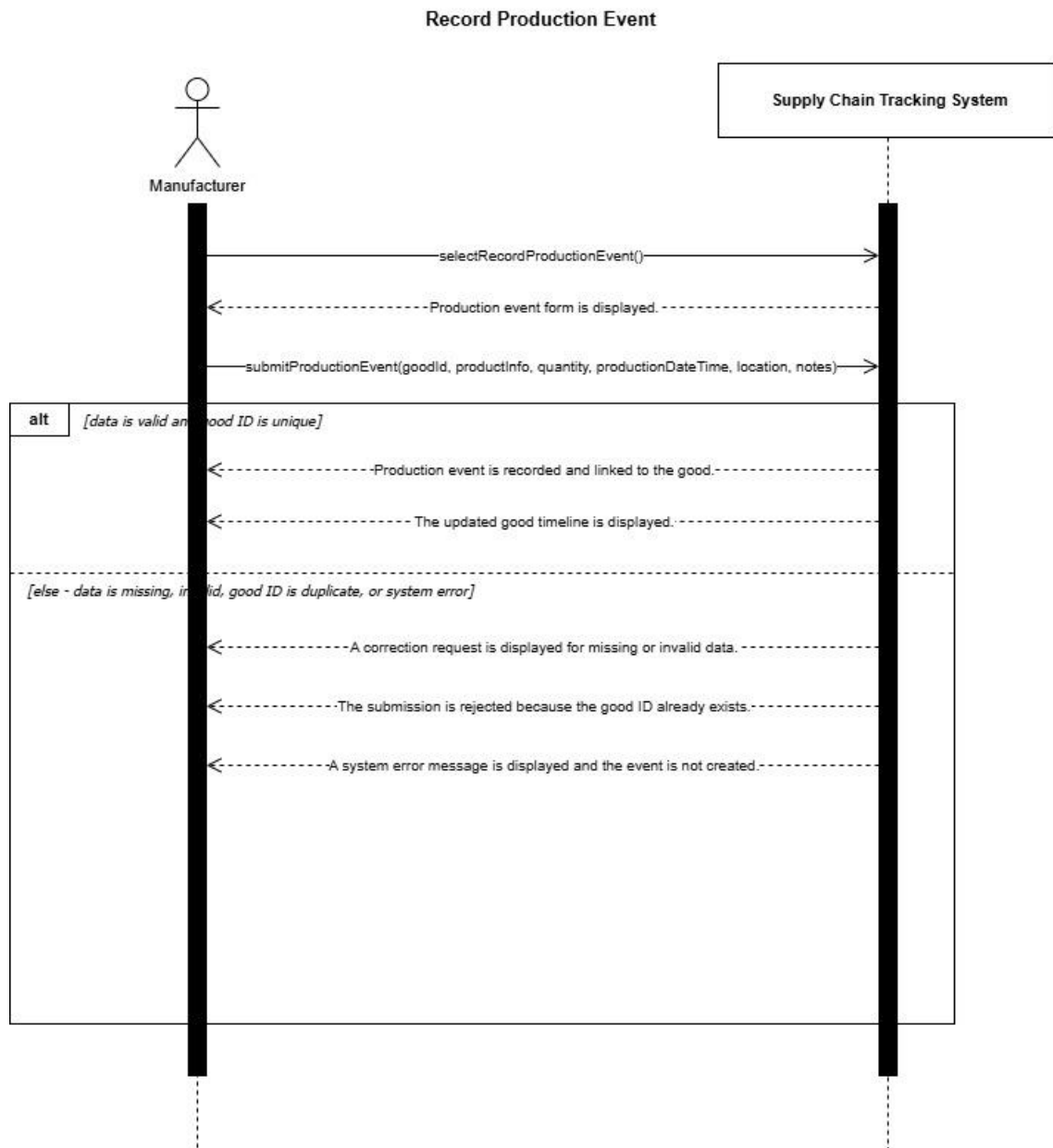


6. System Sequence diagram 6:

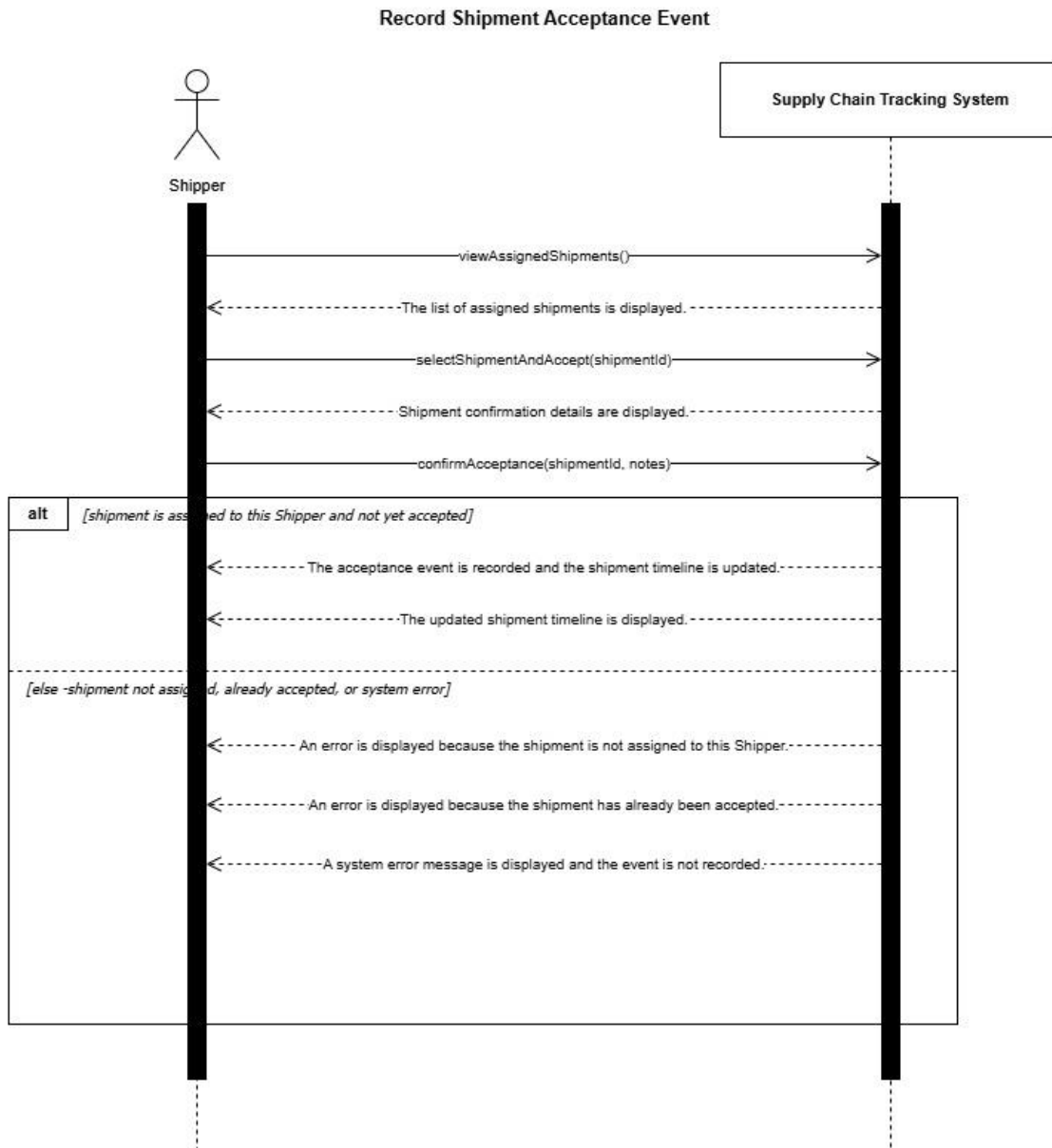
Supply Chain Tracking System: Record Dispatch Event



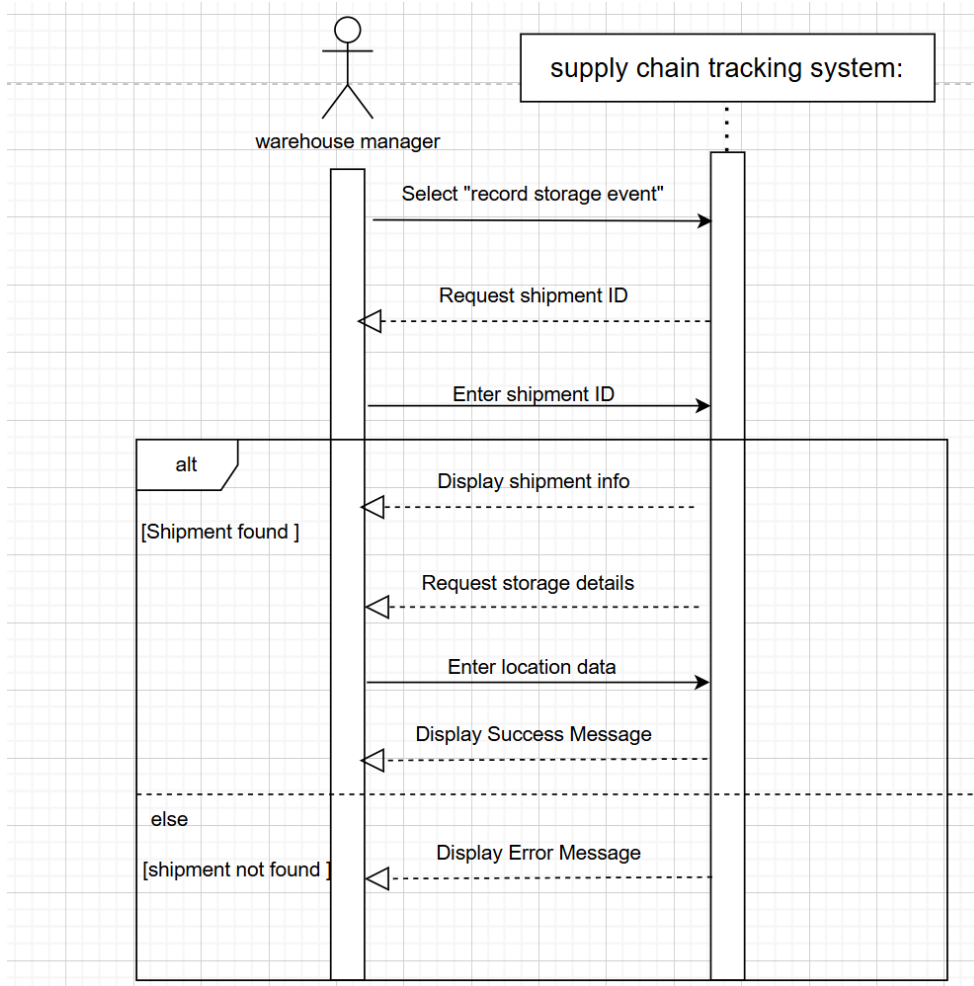
7. System Sequence diagram 7:



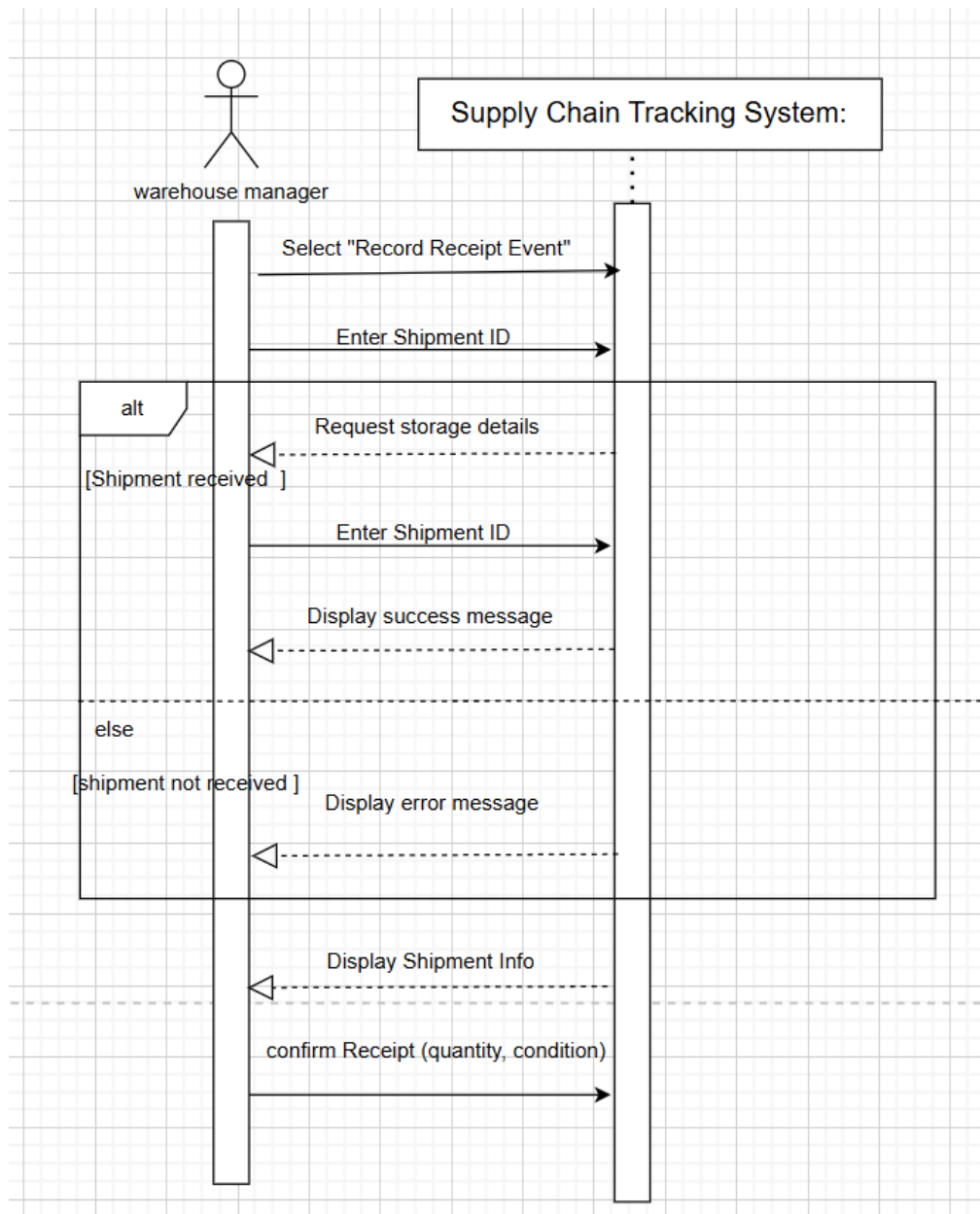
8. System Sequence diagram 8:



9. System Sequence diagram 9:

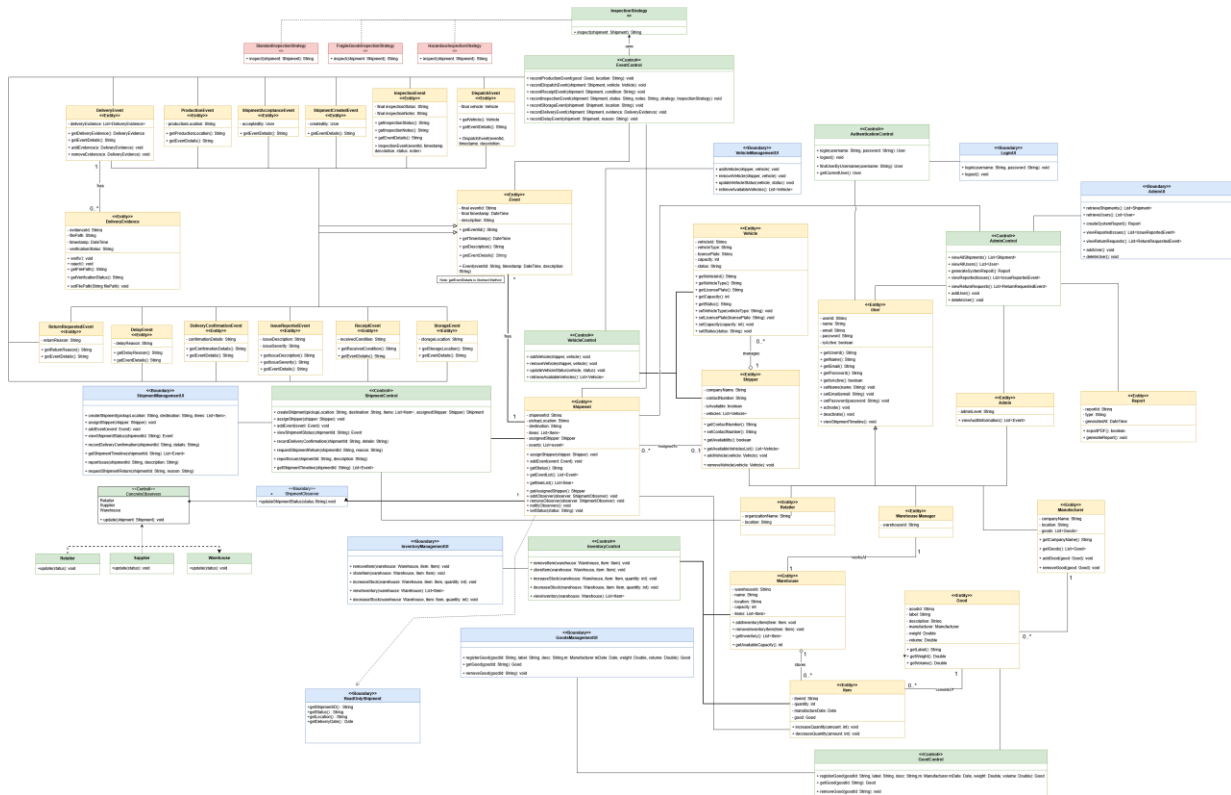


10. System Sequence diagram 10:



5. Full Class Diagram

Image(.png) will also be provided with Submission

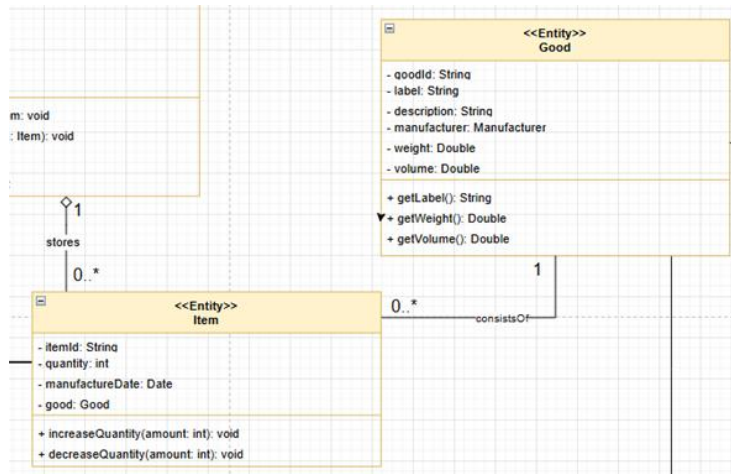


6. Design Patterns

Abstraction-Occurrence Design Pattern

Youssef241784

Screenshot Inside Class Diagram

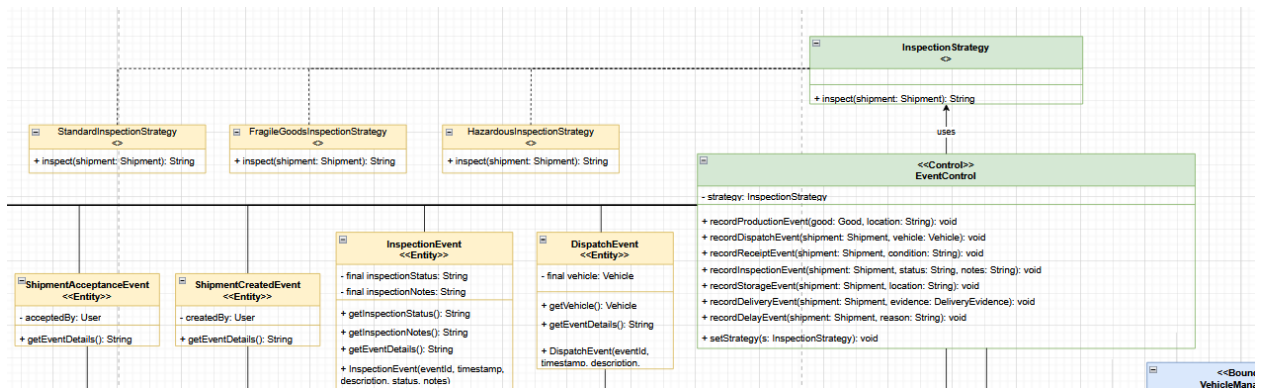


Description & Justification: Abstraction-Occurrence pattern is normally used when there exists a general concept (Abstraction) and many real instances of it (Occurrence). It is applied in the supply chain tracking system in which abstraction is represented as Good, and occurrence is represented as Item. Moreover, good stores information that is the same for all items and Item stores information that is different for each instance. The relationship stated is that one good has many items. To justify the use of this pattern, a few advantages were noted. One of which is that data duplication will be avoided. Secondly, if I need to edit a good, I would have to update it in every item if abstraction-occurrence wasn't one of the design patterns to this system.

Strategy Design Pattern

Zeyad Ashraf 245975

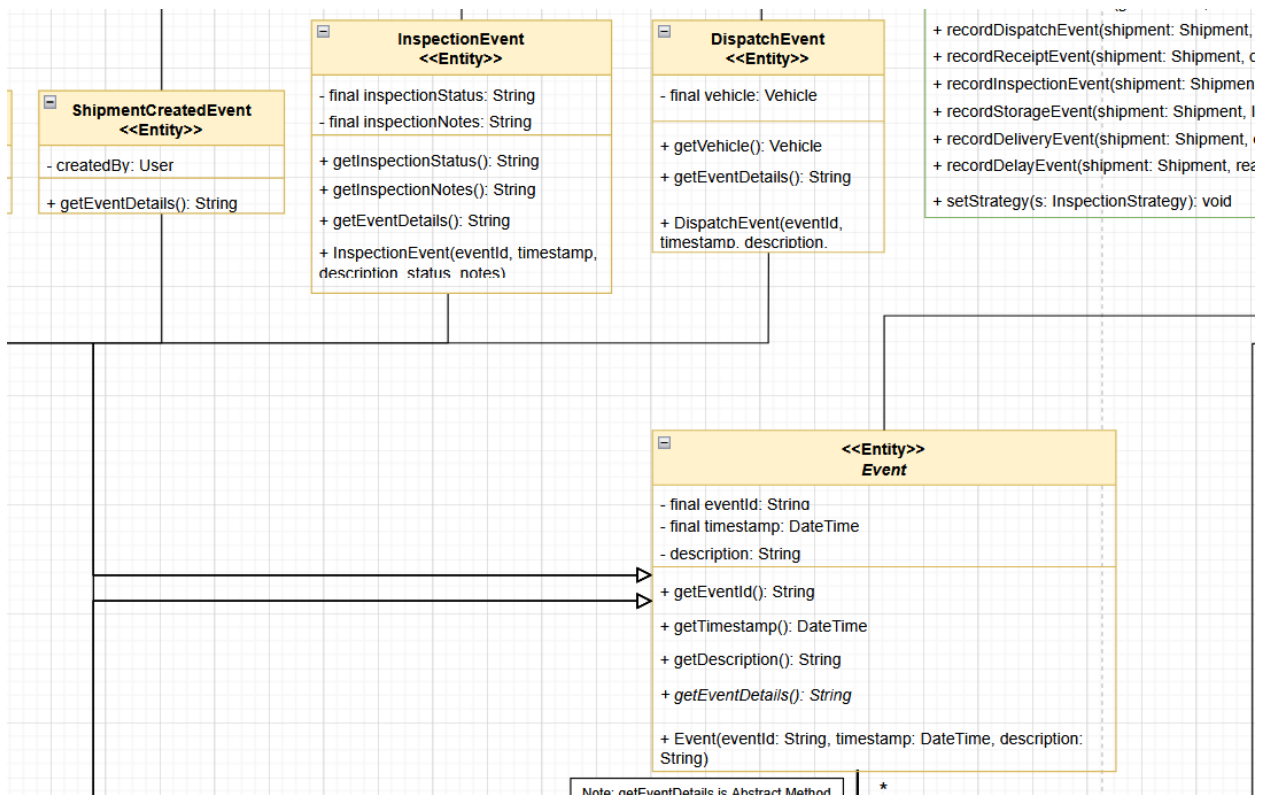
When a warehouse manager records an inspection event, not all shipments are treated the same. A shipment carrying fragile goods needs a more careful inspection than a standard one, and a shipment containing hazardous material requires specific safety checks. Instead of putting all this logic inside EventControl and making a lot of if-else conditions, we use the strategy pattern to separate each inspection approach into its own class. The idea is very simple eventControl holds a reference to an InspectionStrategy, and at runtime the right strategy is used depending on the type of shipment being inspected. According to its type, EventControl just calls inspect() and gets back the result without caring about the details.



Immutable Design Pattern

Ziad Sameh 233802

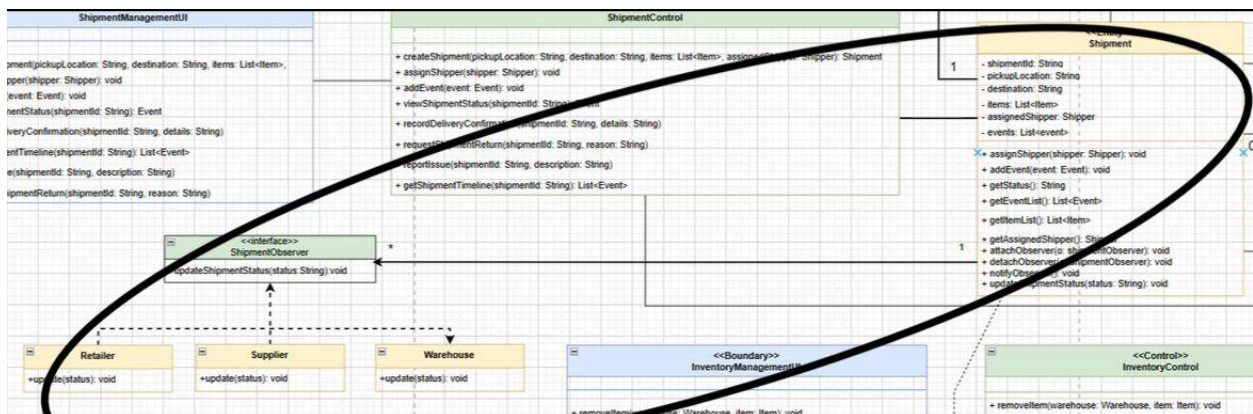
Immutable patterns are core principles in the system because once something happens in the supply chain, it cannot be altered. An inspection result, a dispatch record, a delivery confirmation these are facts that are not changed after they are written. All attributes are marked as final, meaning no setter methods are allowed or exist. Nobody can go back and quietly change anything. This gives the system a trustworthy audit trail that stakeholders can rely on.



Observer design pattern:

Ziyad243886

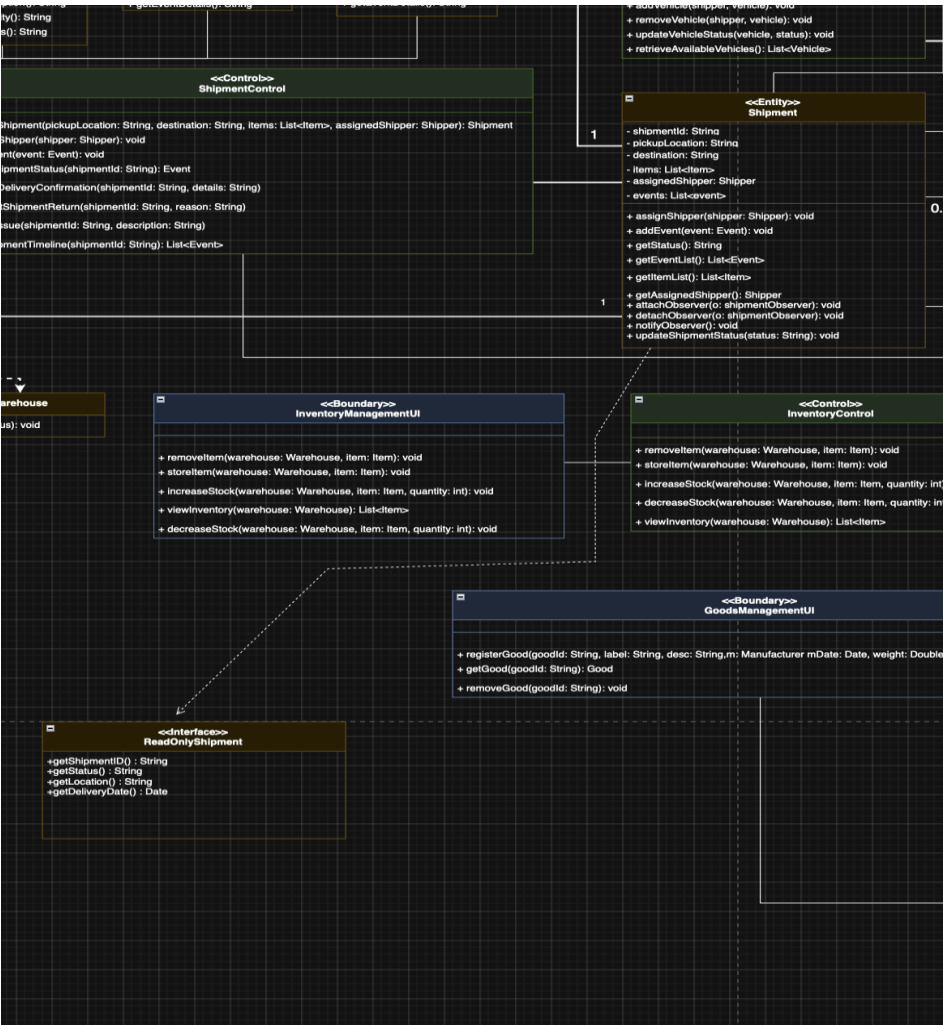
Observer Pattern is applied to the Shipment class to notify multiple system components when shipment status changes. Shipment acts as the subject and maintains a list of observers. Classes such as Retailer, Supplier, and Warehouse implement the ShipmentObserver interface and receive updates through the update() method whenever the shipment state changes. This design reduces coupling and allows new observers to be added easily.



Read only design pattern:

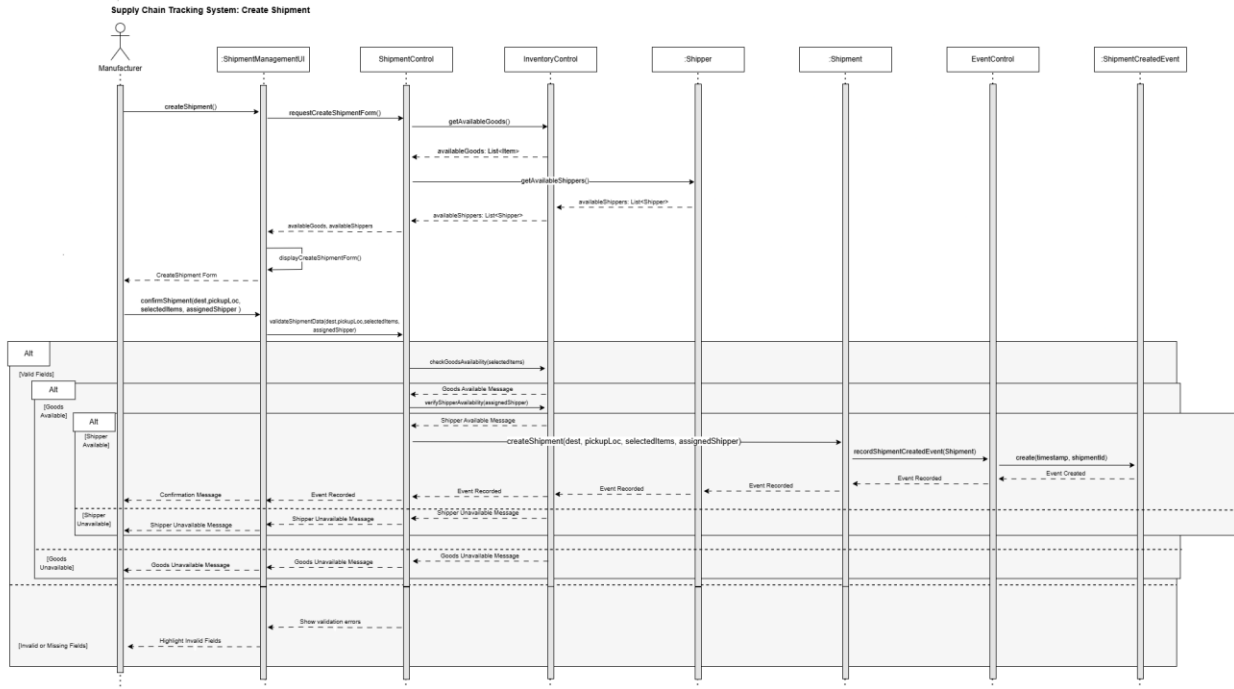
Mostafa240503:

The Read Only Interface Design Pattern is used to control access to an object's data by allowing some components to view it without being able to modify it. This is achieved by defining a separate interface that includes only read operations, while the full class still supports both reading and writing for authorized components. By separating access in this way the pattern helps protect important data from unintended changes and improves the overall reliability and security of the system.

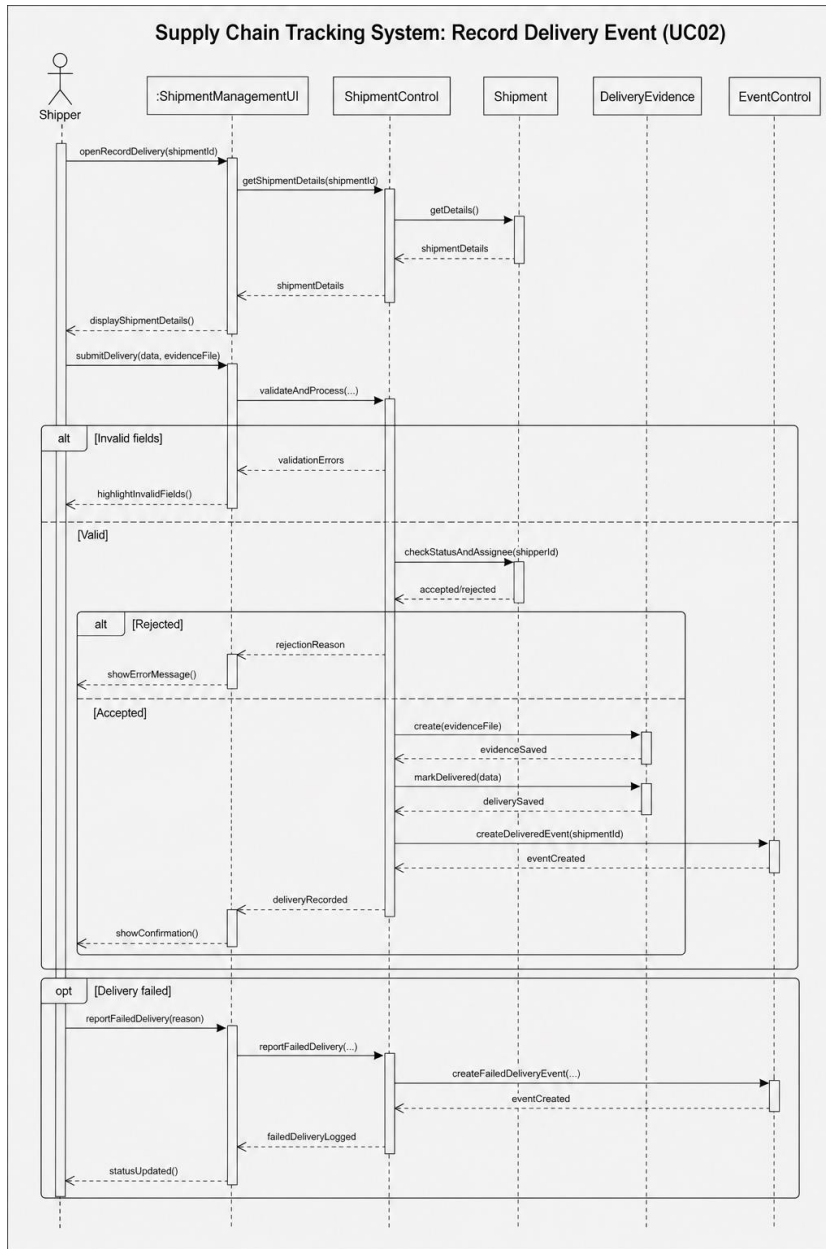


7. Sequence Diagrams

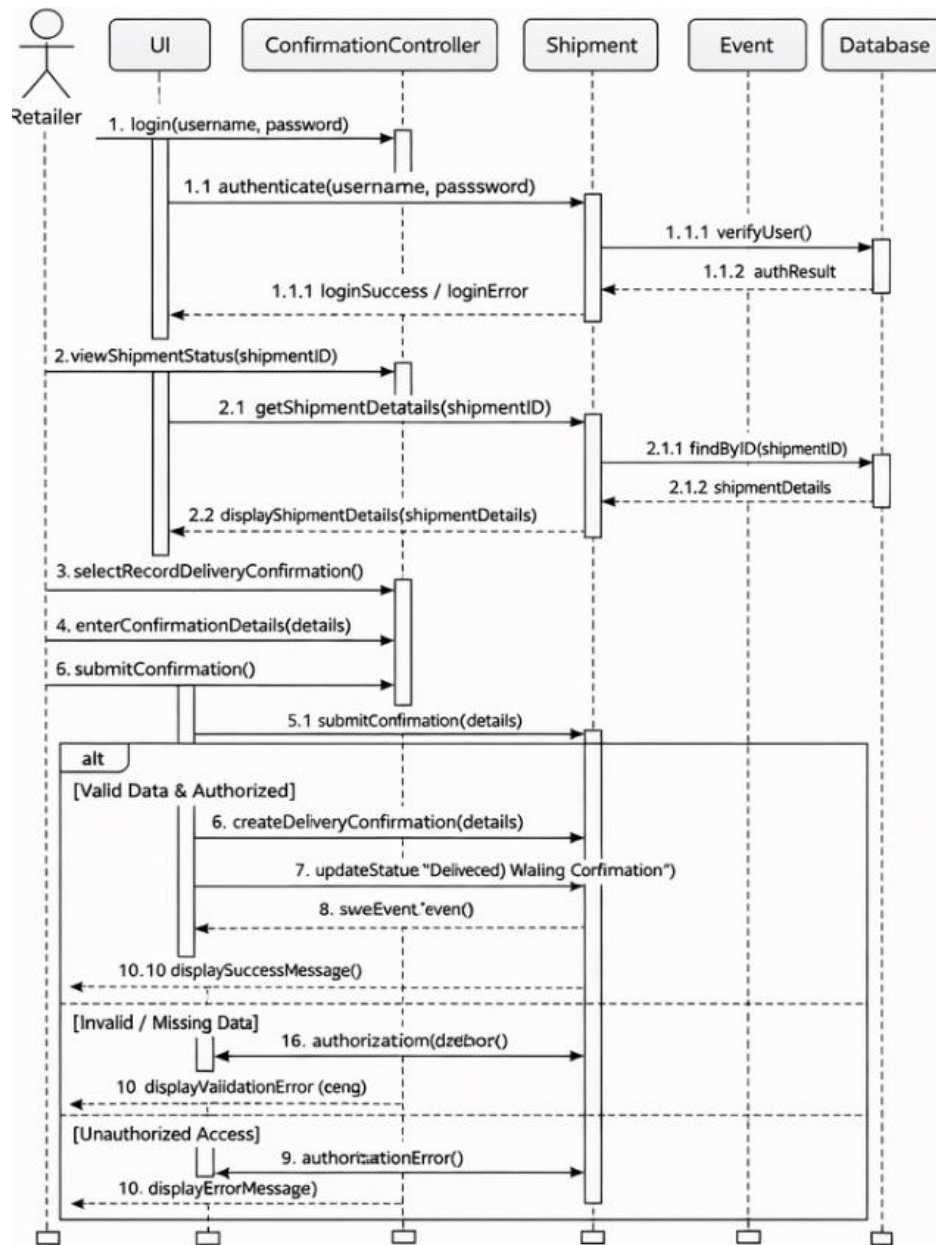
UC01 - Create Shipment [Youssef Alaa]



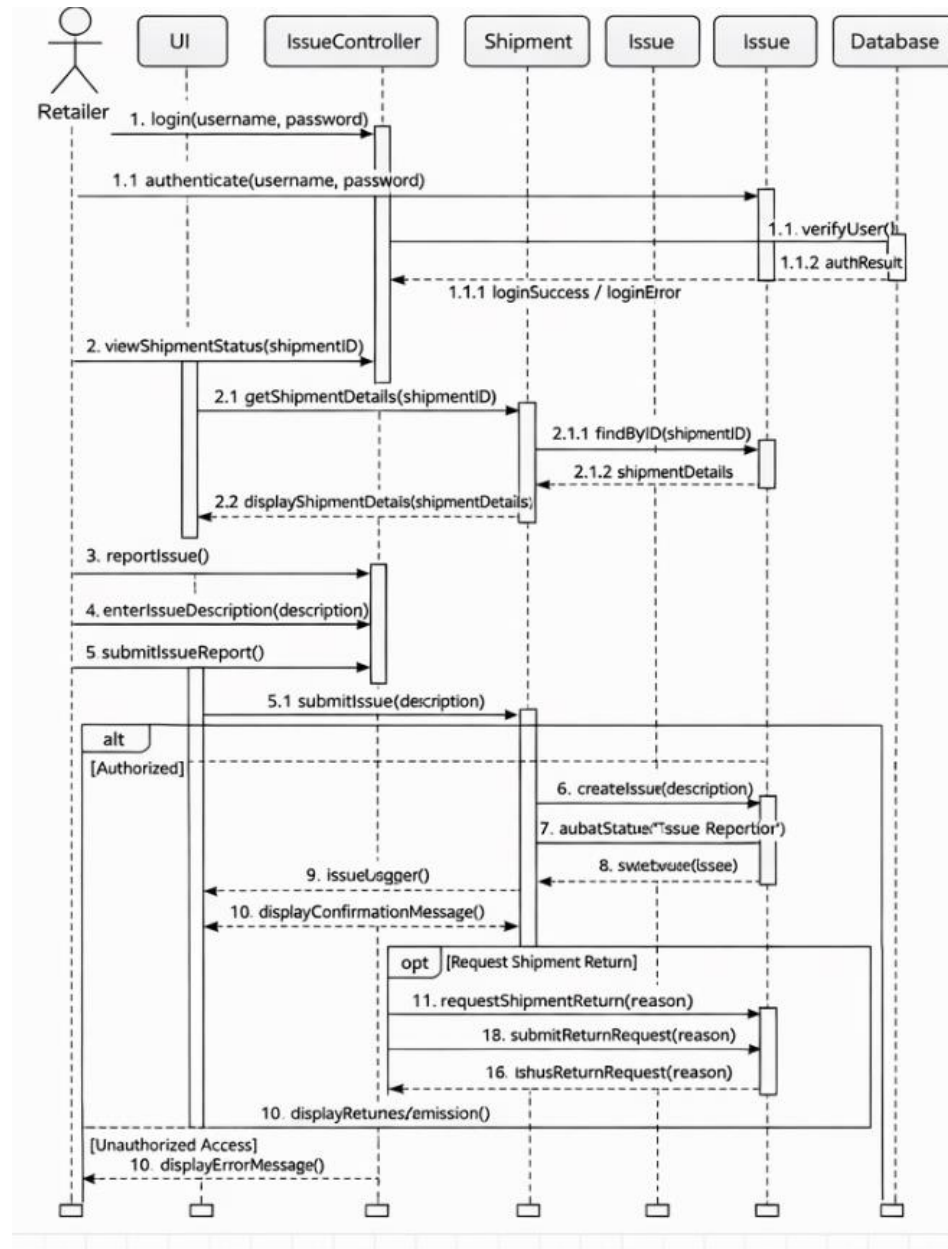
UC02 - Record Delivery Event [Youssef Alaa]



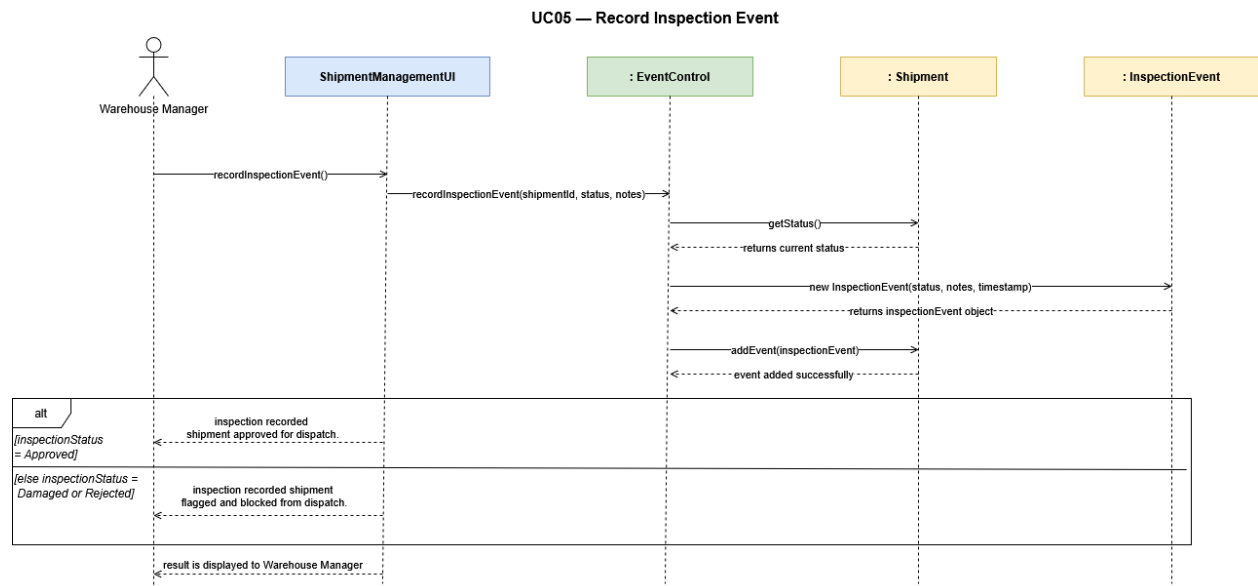
UC03



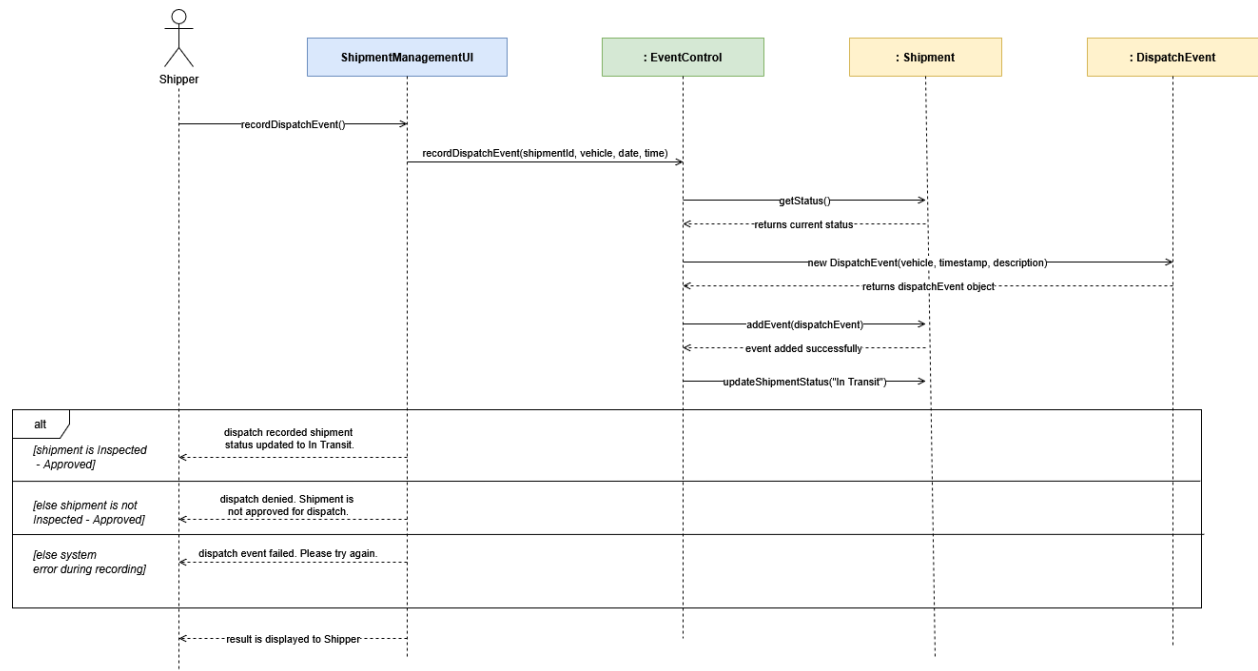
UC04



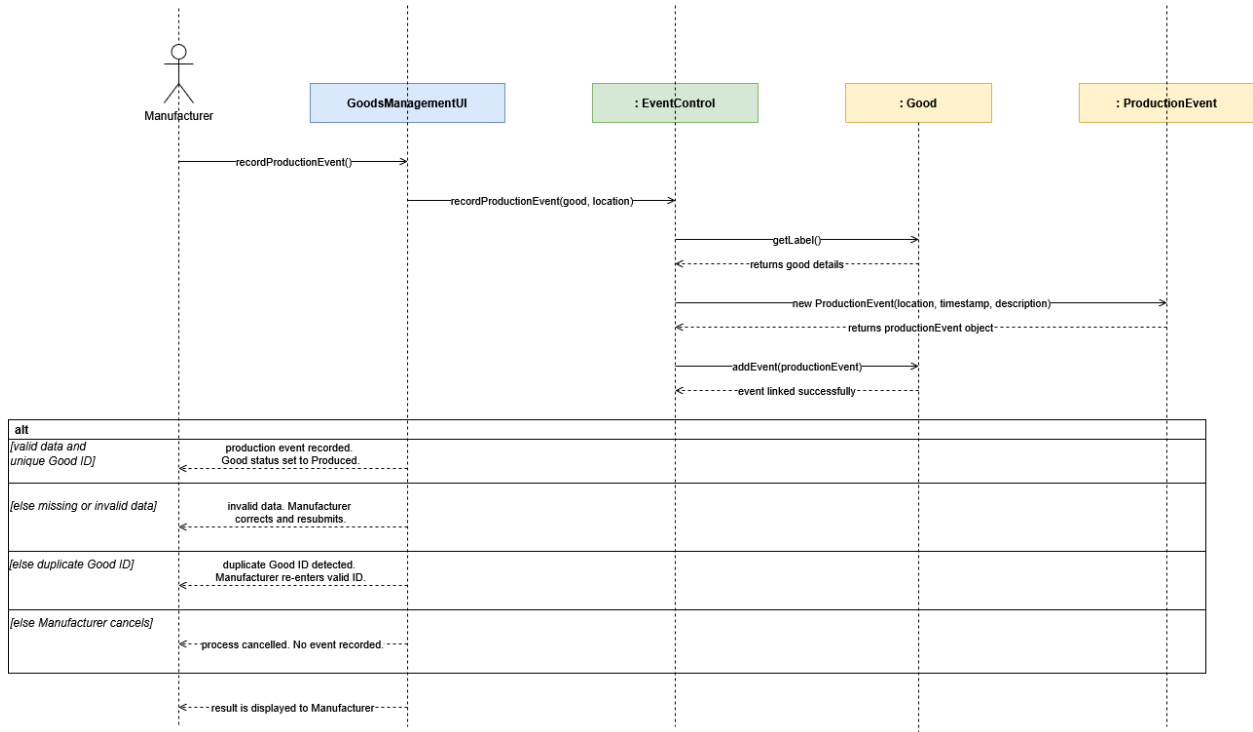
UC05



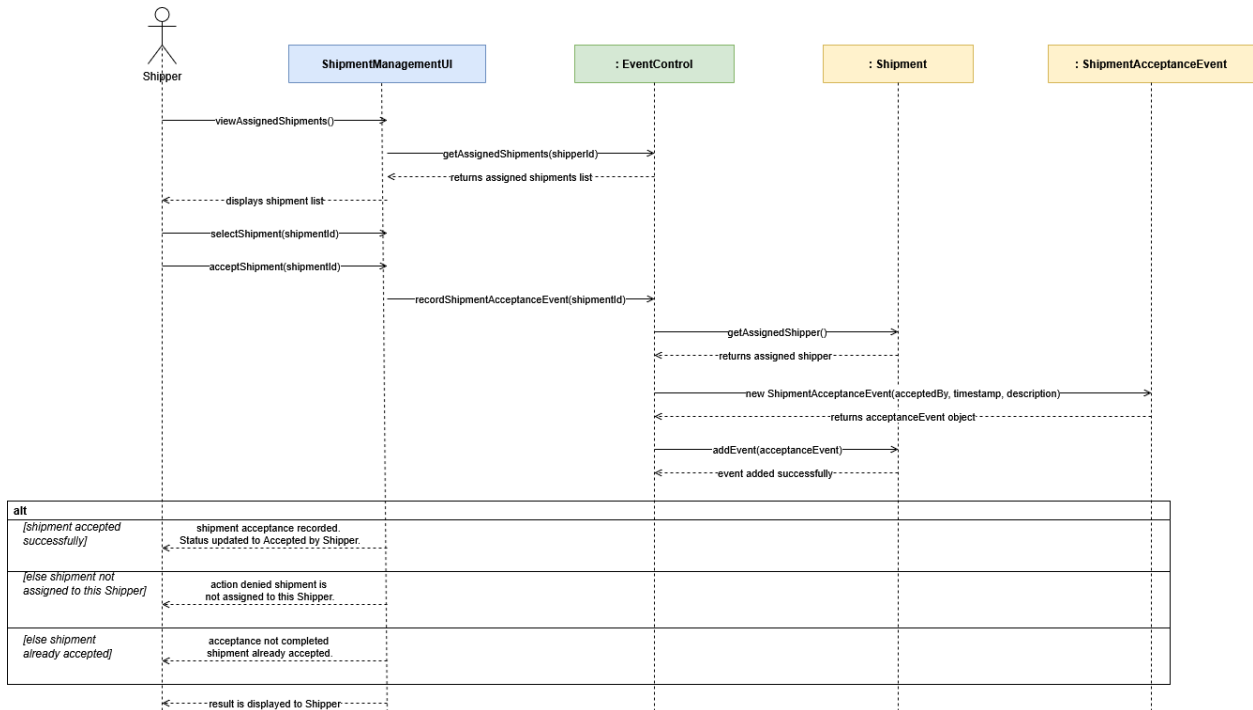
UC06



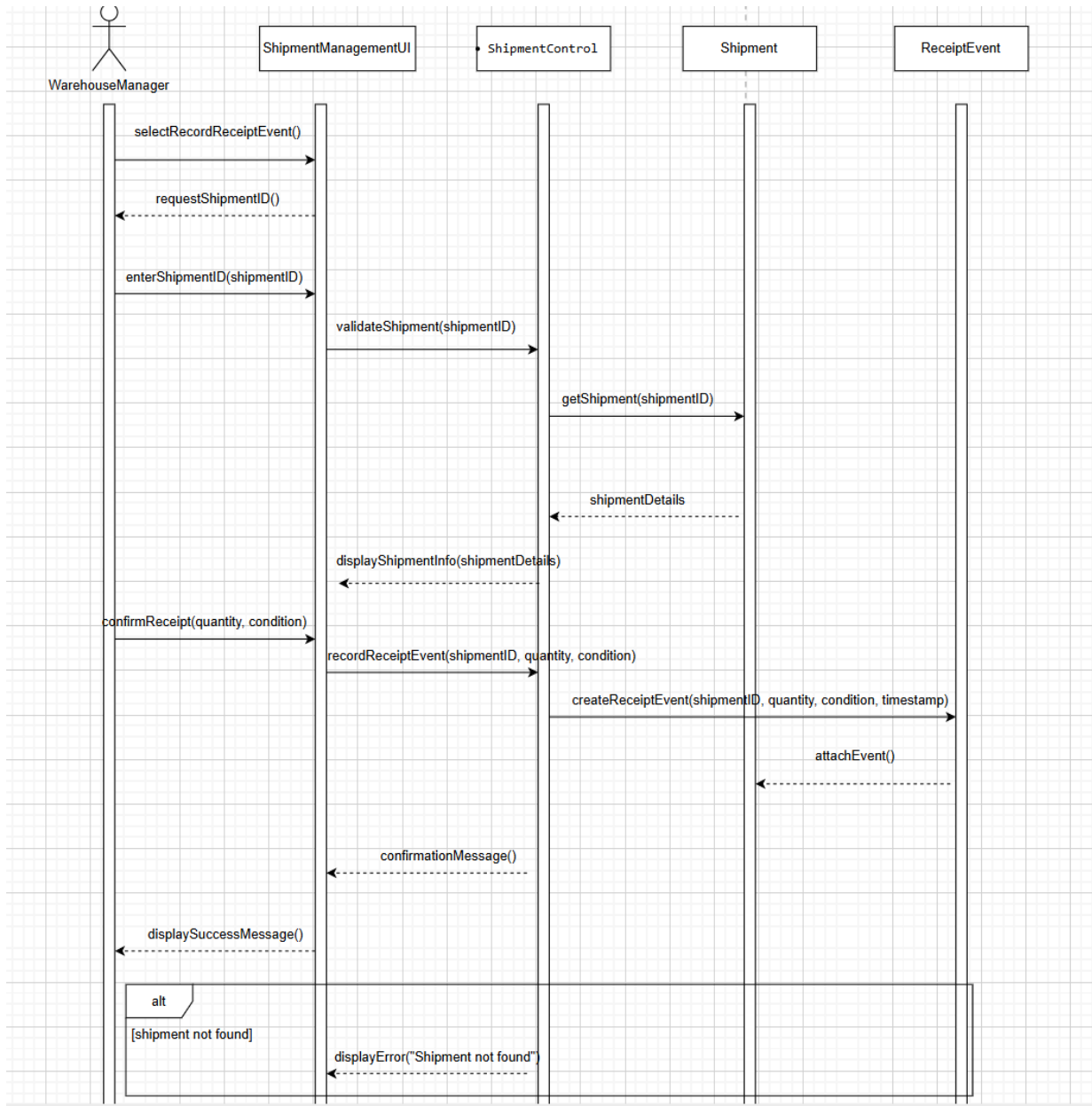
UC07



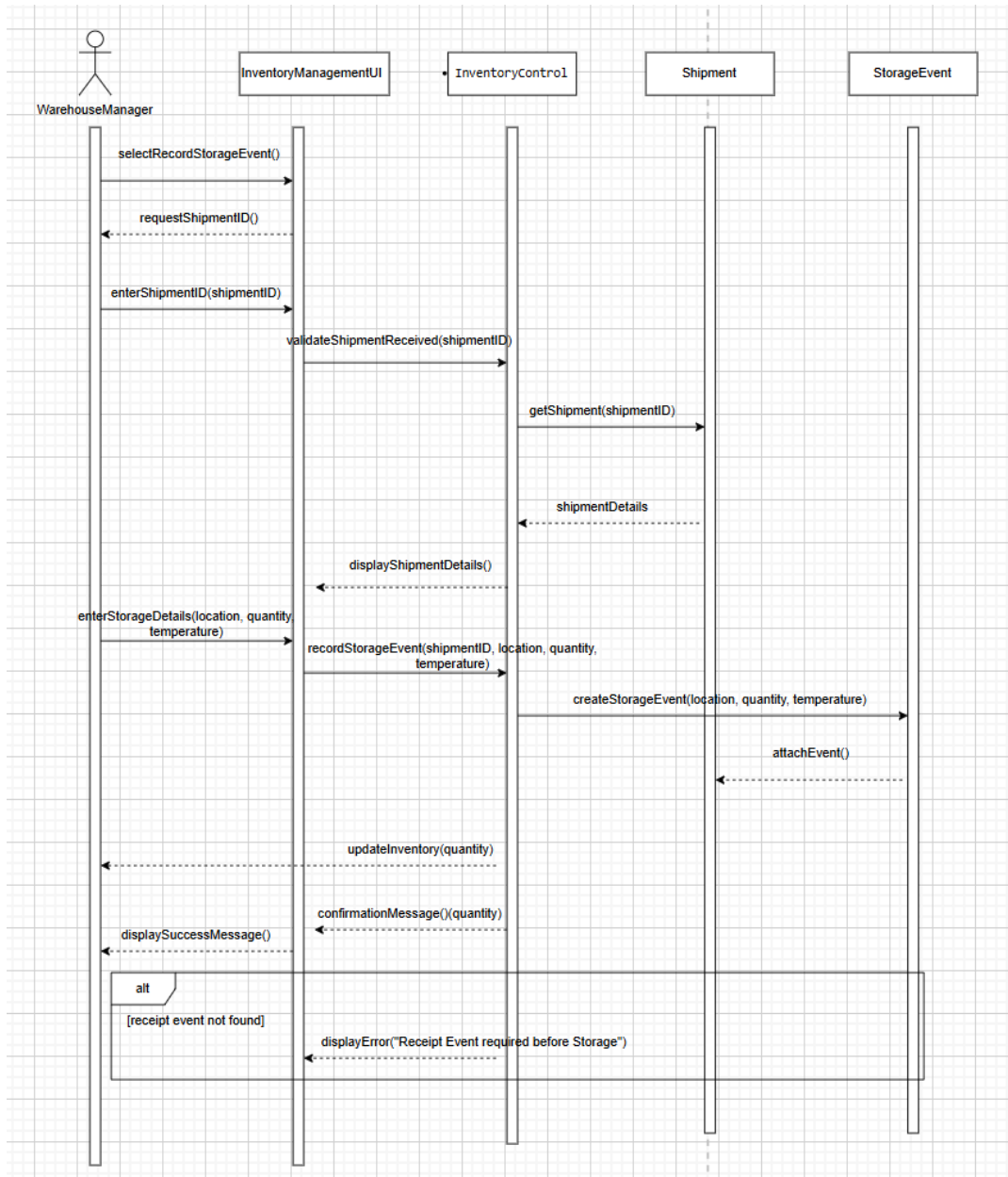
UC08



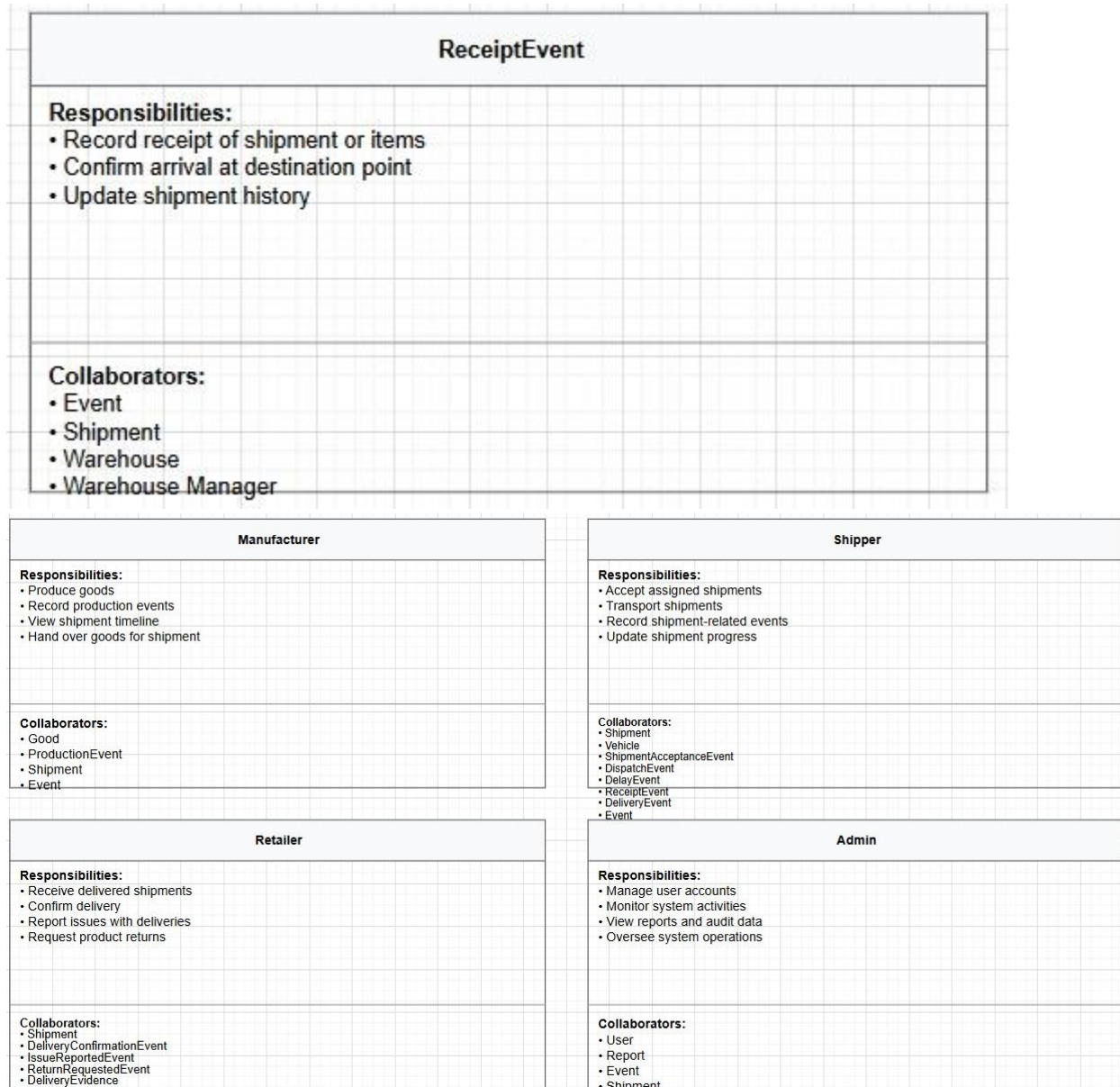
UC09



UC10



8. CRC Cards



Vehicle
Responsibilities: • Represent transport vehicle details • Support shipment transportation • Link dispatched shipments to a vehicle
Collaborators: • Shipper • Shipment • DispatchEvent

ProductionEvent
Responsibilities: • Record production of a good • Store production event details • Preserve immutable production history
Collaborators: • Event • Good • Manufacturer

InspectionEvent
Responsibilities: • Record inspection results • Store inspection notes and status • Support quality and condition tracking
Collaborators: • Event • Shipment • Item • Warehouse Manager

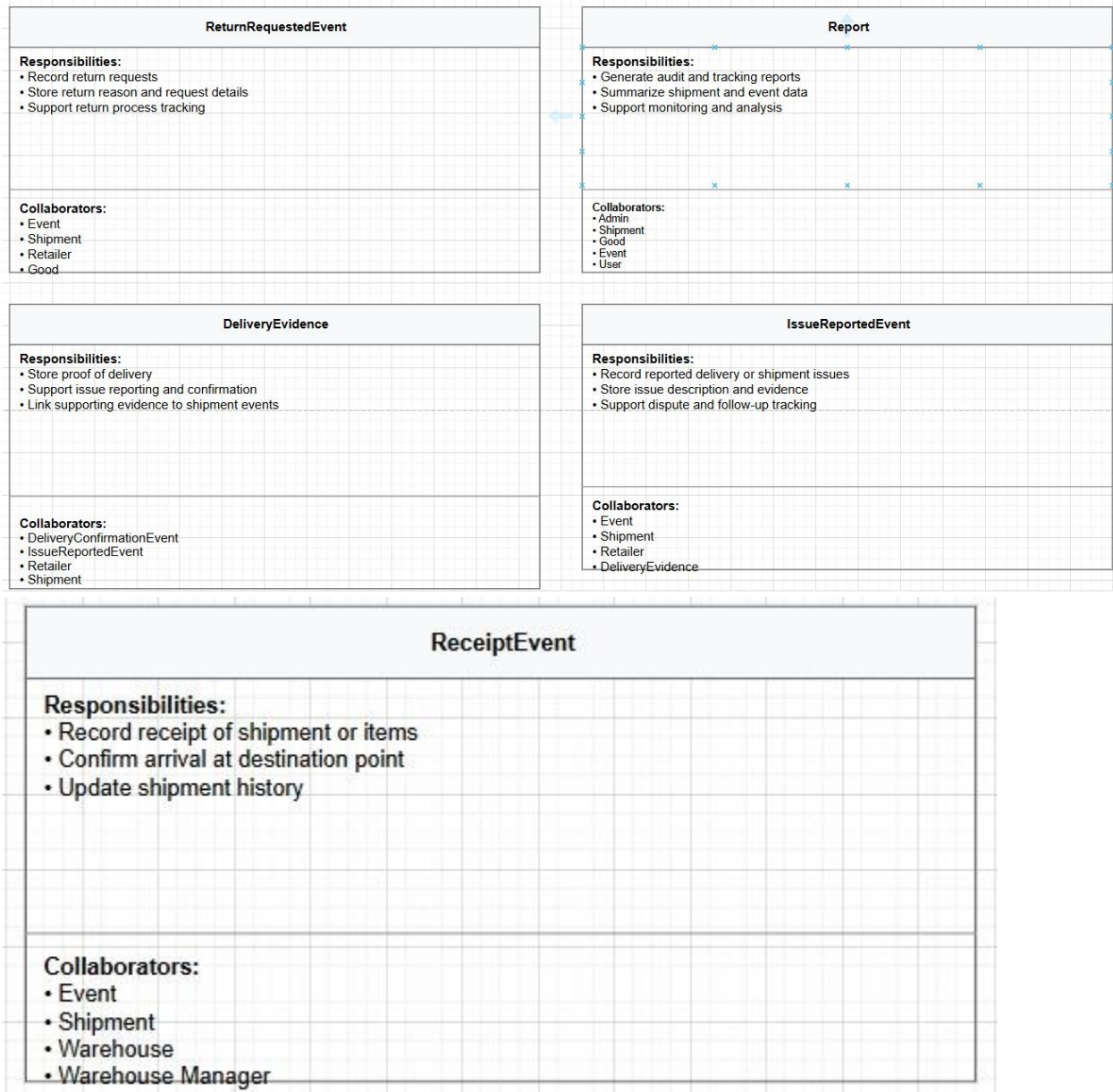
StorageEvent
Responsibilities: • Record storage of items in warehouse • Track item placement and stock updates • Preserve warehouse storage history
Collaborators: • Event • Warehouse • Item • Warehouse Manager

ShipmentAcceptanceEvent
Responsibilities: • Record shipper acceptance of a shipment • Confirm transport responsibility • Update shipment timeline
Collaborators: • Event • Shipment • Shipper

DelayEvent
Responsibilities: • Record shipment delay information • Store reason for delay • Support tracking of shipment exceptions
Collaborators: • Event • Shipment • Shipper

DeliveryEvent
Responsibilities: • Record delivery of shipment • Store delivery status and time • Update shipment progress to delivered
Collaborators: • Event • Shipment • Shipper • Retailer

DeliveryConfirmationEvent
Responsibilities: • Confirm successful delivery • Store confirmation details • Finalize proof of delivery in event history
Collaborators: • Event • Shipment • Retailer • DeliveryEvidence



Warehouse Manager
Responsibilities: • Manage warehouse operations • Store received items • Track stock movement • Monitor warehouse inventory
Collaborators: • Warehouse • Item • StorageEvent • InspectionEvent • ReceiptEvent

Item
Responsibilities: • Represent a shipment item • Store item quantity and details • Link goods to shipments • Support inventory tracking
Collaborators: • Good • Shipment • Warehouse

DispatchEvent
Responsibilities: • Record shipment dispatch • Store dispatch details and timestamp • Link shipment movement to a vehicle
Collaborators: • Event • Shipment • Vehicle • Shipper

ShipmentCreatedEvent
Responsibilities: • Record creation of a shipment • Store initial shipment details • Start shipment event history
Collaborators: • Event • Shipment • Item

Shipment	
Responsibilities	Collaborators
- Track shipment status - Generate shipment ID - Store shipment goods - maintain shipment event history - maintain shipment information	Event Warehouse Shipper Good

Class: Good Responsibilities: Track product lifecycle maintain production information store product details link goods to shipments Collaborators: ProductionEvent shipment

Class: Warehouse

Responsibilities:

Inspect goods
store goods in warehouse
update inventory
receive shipments

Collaborators:

Good
Shipment
Event

Class: Event

Responsibilities:

Store event type
record event timestamp
link events to shipments or goods
maintain immutable history

Collaborators:

Good
Shipments

9. Testing Reports

Report 1

Test Case ID	TC01_InspectionStrategy
Description	To verify that strategy pattern returns actual correct inspection outputs
Preconditions	• Shipment object exists
Postconditions	• Standard will output INSPECTED_APPROVED • Fragile will output INSPECTED_APPROVED_FRAGILE_HANDLING • Hazardous will output INSPECTED_APPROVED_HAZMAT_PROTOCOL
Main Path	1. Create Shipment 2. Execute StandardInspectStrategy.inspect 3. Execute FragileGoodsInspectionStrategy.inspect 4. Execute HazardousInspectionStrategy.inspect 5. Assert all expected outputs
Alternative Path	Strategy type that is unsupported shouldn't be used.

Screenshots

The screenshot displays the test results for the `InspectionStrategyTest` class. A green bar at the top indicates "Tests passed: 100.00 %". Below this, a summary states "Both tests passed. (0.131 s)". The test log shows the execution of `testUnsupportedStrategyIsRejected` and `testInspectionStrategies`, both of which completed successfully. The source code for the test class is shown below the log, featuring two test methods: `testInspectionStrategies` which verifies the output of three inspection strategies (Standard, Fragile, and Hazardous) against expected strings, and `testUnsupportedStrategyIsRejected` which verifies that an `IllegalArgumentException` is thrown when an unsupported strategy is set.

```
@Before
public void setUp() {
    shipment = new Shipment("S1", "A", "B", Collections.emptyList());
}

@Test
public void testInspectionStrategies() {
    String standard = new StandardInspectionStrategy().inspect(shipment);
    String fragile = new FragileGoodsInspectionStrategy().inspect(shipment);
    String hazardous = new HazardousInspectionStrategy().inspect(shipment);

    assertEquals("INSPECTED_APPROVED", standard);
    assertEquals("INSPECTED_APPROVED_FRAGILE_HANDLING", fragile);
    assertEquals("INSPECTED_APPROVED_HAZMAT_PROTOCOL", hazardous);
}

@Test
public void testUnsupportedStrategyIsRejected() {
    EventControl control = new EventControl(null);
    try {
        control.setStrategy(null);
        fail("Expected IllegalArgumentException");
    } catch (IllegalArgumentException ex) {
        assertEquals("Inspection strategy is required.", ex.getMessage());
    }
}
```


Report 2

Test Case ID	TC02_ShipmentStatus
Description	This is to verify that shipment status is derived from latest event in Timeline
Preconditions	A shipment object is actually existing and has initial CREATED state & one item.
Postconditions	- Status changes correctly to IN_TRANSIT. - Status then changes to RECEIVED_AT_WAREHOUSE. - Status then changes to DELIVERED.
Main Path	- Create shipment object - Add DispatchEvent, assert status IN_TRANSIT. - Add ReceiptEvent, assert status RECEIVED_AT_WAREHOUSE. - Add DeliveryEvent, assert status DELIVERED.
Alternative Path	If events are added in an incorrect order, then the transition of status should be rejected by the implemented rules.

Screenshots

```
@Before
public void setUp() {
    Manufacturer manufacturer = new Manufacturer("M1", "M", "m@m.com", "x", true, "Co");
    Good good = new Good("G1", "Label", "Desc", manufacturer, 10, 2);
    Item item = new Item("I1", 10, LocalDate.now(), good);
    shipment = new Shipment("S1", "Factory", "Retailer", List.of(item));
}

@Test
public void testStatusDerivedFromLastEvent() {
    shipment.addEvent(new DispatchEvent("E1", LocalDateTime.now(), "Dispatch", "V1"));
    assertEquals("IN_TRANSIT", shipment.getStatus());

    shipment.addEvent(new ReceiptEvent("E2", LocalDateTime.now(), "Receipt", "GOOD"));
    assertEquals("RECEIVED_AT_WAREHOUSE", shipment.getStatus());

    DeliveryEvidence ev = new DeliveryEvidence("D1", "path", LocalDateTime.now(), "UNVERIFIED");
    shipment.addEvent(new DeliveryEvent("E3", LocalDateTime.now(), "Delivery", ev));
    assertEquals("DELIVERED", shipment.getStatus());
}

@Test
public void testInvalidEventOrderIsRejected() {
    try {
        Event.assertCanRecordEvent(Role.SHIPPER, "DELIVERY", shipment.getStatus());
        fail("Expected IllegalArgumentException");
    } catch (IllegalArgumentException ex) {
        assertEquals("CREATED", shipment.getStatus());
    }
}
```

Tests passed: 100.00 %	Running ShipmentStatusTest class tests...
Both tests passed. (0.128 s)	Done: testStatusDerivedFromLastEvent
	Done: testInvalidEventOrderIsRejected
	Finished ShipmentStatusTest class tests.

Report 3

Test Case ID	TC03_ItemQuantity
Description	To verify that Item quantity updates in a correct way where increasing and decreasing is accepted while any invalid quantity is rejected
Preconditions	• A valid Good object is existing and is linked to the Item. • An item instance is created with a known initial quantity.
Postconditions	• Quantity when increased is updated correctly: quantity + a • Quantity when decreased is updated correctly: quantity – a • If invalid operation is detected, IllegalArgumentException is thrown. • On any failed operation the quantity will stay the same as the pre-call value. • Quantity will never be in negative value.
Main Path	1. Create item with an initial quantity of 10 2. Call increaseQuantity(5). 3. Verify that quantity is 15. 4. Call decreaseQuantity(3). 5. Verify that quantity is 12.
Alternative Path	1. Create Item with initial quantity of 10 2. Call increaseQuantity(0) 3. Expect IllegalArgumentException 4. Verify quantity is still 10 5. Call decreaseQuantity(0) 6. Expect IllegalArgumentException 7. Verify that quantity is unchanged afterwards. 8. Call decreaseQuantity(11) [Greater than Current Quantity] 9. Expect IllegalArgumentException 10. Verify that quantity is unchanged afterwards. 11. Call increaseQuantity(-1) 12. Expect IllegalArgumentException 13. Verify that quantity is unchanged afterwards. 14. Call decreaseQuantity(-1) 15. Expect IllegalArgumentException 16. Verify that Quantity is unchanged afterwards.

Screenshots

Tests passed: 100.00 %	Running ItemQuantityTest class tests...
All 6 tests passed. (0.708 s)	Done: testDecreaseQuantityZeroThrowsAndKeepsState
	Done: testIncreaseQuantityNegativeThrowsAndKeepsState
	Done: testDecreaseQuantityNegativeThrowsAndKeepsState
	Done: testIncreaseAndDecreaseItemQuantity
	Done: testIncreaseQuantityZeroThrowsAndKeepsState
	Done: testDecreaseQuantityGreaterThanCurrentThrowsAndKeepsState
	Finished ItemQuantityTest class tests.

```
public class ItemQuantityTest {
    private Manufacturer manufacturer;
    private Good good;

    @Before
    public void setUp() {
        manufacturer = new Manufacturer("M1", "M", "m@m.com", "x", true, "Co");
        good = new Good("G1", "Label", "Desc", manufacturer, 10, 2);
    }

    @Test
    public void testIncreaseAndDecreaseItemQuantity() {
        Item item = new Item("I1", 10, LocalDate.now(), good);

        item.increaseQuantity(5);
        assertEquals(15, item.getQuantity());

        item.decreaseQuantity(3);
        assertEquals(12, item.getQuantity());
    }

    @Test
    public void testIncreaseQuantityZeroThrowsAndKeepsState() {
        Item item = createItemWithQuantity(10);
        int before = item.getQuantity();

        try {
            item.increaseQuantity(0);
            fail("Expected IllegalArgumentException");
        } catch (IllegalArgumentException ex) {
            assertEquals(before, item.getQuantity());
        }
    }
}
```

```
@Test
public void testIncreaseQuantityNegativeThrowsAndKeepsState() {
    Item item = createItemWithQuantity(10);
    int before = item.getQuantity();

    try {
        item.increaseQuantity(-1);
        fail("Expected IllegalArgumentException");
    } catch (IllegalArgumentException ex) {
        assertEquals(before, item.getQuantity());
    }
}

@Test
public void testDecreaseQuantityZeroThrowsAndKeepsState() {
    Item item = createItemWithQuantity(10);
    int before = item.getQuantity();

    try {
        item.decreaseQuantity(0);
        fail("Expected IllegalArgumentException");
    } catch (IllegalArgumentException ex) {
        assertEquals(before, item.getQuantity());
    }
}
```

```

@Test
public void testDecreaseQuantityNegativeThrowsAndKeepsState() {
    Item item = createItemWithQuantity(10);
    int before = item.getQuantity();

    try {
        item.decreaseQuantity(-1);
        fail("Expected IllegalArgumentException");
    } catch (IllegalArgumentException ex) {
        assertEquals(before, item.getQuantity());
    }
}

@Test
public void testDecreaseQuantityGreaterThanCurrentThrowsAndKeepsState() {
    Item item = createItemWithQuantity(10);
    int before = item.getQuantity();

    try {
        item.decreaseQuantity(11);
        fail("Expected IllegalArgumentException");
    } catch (IllegalArgumentException ex) {
        assertEquals(before, item.getQuantity());
    }
}

private Item createItemWithQuantity(int quantity) {
    return new Item("Il", quantity, LocalDate.now(), good);
}

```